

Rapport de projet de fin de module

FICHE DE SYNTHÈSE ACADÉMIQUE

Du perceptron aux modèles séquentiels : exploration des architectures fondamentales du Deep Learning avec PyTorch

Encadré par :

Pr.Mossab Batal

Réalisée par :

Belhachmi Hajar

Module :

Deep Learning

Filière :

Intelligence Artificielle et Science des Données

Année universitaire : 2025-2026

Résumé

Ce projet a été réalisé dans le cadre d'un module académique de **Deep Learning** afin d'étudier, d'implémenter et de comparer plusieurs architectures majeures de réseaux de neurones à l'aide de **PyTorch**. À travers une approche progressive, le travail explore successivement les perceptrons multicouches (MLP), les réseaux de neurones convolutionnels (CNN) et les réseaux de neurones récurrents (RNN), chacun étant appliqué à une problématique adaptée à sa nature.

La première partie est consacrée aux MLP et met en évidence les principes fondamentaux de l'apprentissage supervisé sur des données tabulaires. La seconde partie aborde les CNN à travers une tâche de classification d'images sur le jeu de données Fashion-MNIST, en étudiant notamment les opérations de convolution, de pooling et la visualisation des caractéristiques apprises. Enfin, la troisième partie traite des modèles séquentiels, depuis les RNN simples jusqu'aux architectures LSTM, GRU et Seq2Seq, appliquées à une tâche de traduction automatique.

Au-delà de l'implémentation des modèles, ce projet accorde une place importante à l'analyse des résultats, à l'expérimentation des hyperparamètres et à la comparaison des performances obtenues. Il permet ainsi de mieux comprendre les forces, les limites et les domaines d'application de chaque famille d'architectures, tout en développant des compétences pratiques en Deep Learning et en programmation avec PyTorch.

Abstract

This project was carried out as part of an academic **Deep Learning** course with the aim of studying, implementing, and comparing several fundamental neural network architectures using the **PyTorch** framework. Following a progressive learning approach, the work explores three major families of deep learning models: Multi-Layer Perceptrons (MLP), Convolutional Neural Networks (CNN), and Recurrent Neural Networks (RNN).

The first part focuses on MLPs and introduces the core principles of supervised learning applied to tabular data. The second part investigates CNNs through an image classification task using the Fashion-MNIST dataset, emphasizing convolution operations, pooling mechanisms, and the visualization of learned feature representations. The final part addresses sequential modeling by examining vanilla RNNs, LSTMs, GRUs, and Seq2Seq architectures within the context of machine translation tasks.

Beyond model implementation, this project highlights the importance of experimental analysis, hyperparameter exploration, and performance evaluation. The comparative study provides valuable insights into the strengths, limitations, and application domains of each architecture. Overall, this work contributes to a deeper understanding of deep learning concepts while strengthening practical skills in model development and experimentation with PyTorch.

Table des matières

Résumé	2
Abstract	3
Table des matières	4
Listes des figures	8
Listes des tableaux.....	9
Listes des abréviations.....	10
Introduction générale.....	11
1. Chapitre 1 : Étude des perceptrons multicouches (MLP).....	12
1.2.1. nn.Module — La classe de base de tout modèle.....	13
1.2.2. Paramètres et Gradients	13
1.2.3. Propagation Avant et Rétropropagation.....	14
1.2.4. Gestion du Device (CPU / GPU)	14
1.3.1. Présentation du Dataset Breast Cancer Wisconsin.....	14
1.3.2. Pipeline Complet de Préparation des Données	15
1.4.1. Version A — nn.Sequential : Architecture Déclarative.....	16
1.4.2. Version B — Classe Personnalisée : Architecture Contrôlée	17
1.5.1. named_parameters() — Inventaire des Paramètres Apprenables	18
1.5.2. state_dict() — État Complet du Modèle	18
1.6.1. Initialisation Gaussienne	19
1.6.2. Initialisation Constante — Brise la Symétrie.....	19
1.6.3. Initialisation Xavier Uniform — Recommandée	20
1.7.1. Fonction de Perte : CrossEntropyLoss.....	20
1.7.2. Optimiseur : Adam.....	21
1.7.3. Tableau des Hyperparamètres et Justifications	21
1.7.4. Boucle d'Entraînement Complète.....	21
1.8.1. Sauvegarde du Meilleur Modèle (Best Checkpoint).....	22
1.8.2. Rechargement et Vérification d'Identité	23
1.9.1. Métriques de Classification Binaire	23
1.9.2. Code d'Évaluation Complet	24
1.9.3. Résultats Expérimentaux sur Breast Cancer Wisconsin	24
1.10.1. Pertinence Théorique : Le Théorème d'Approximation Universelle	26

1.10.2.	Choix Méthodologiques Justifiés	26
1.10.3.	Résultats Expérimentaux et Comparaison	27
1.10.4.	Limites Principales.....	27
2.	Chapitre 2 : Réseaux de neurones convolutionnels (CNN)	28
2.2.1.	Présentation du Dataset.....	29
2.2.2.	Classes du Dataset.....	30
2.2.3.	Chargement et Normalisation	30
2.3.1.	Explosion du Nombre de Paramètres	31
2.3.2.	Perte de la Structure Spatiale	31
2.3.3.	Absence d'Invariance aux Translations.....	31
2.3.4.	Tableau Comparatif MLP vs CNN	32
2.4.1.	Localité (Local Connectivity)	32
2.4.2.	Partage des Poids (Weight Sharing).....	32
2.4.3.	Hierarchie des Représentations.....	33
2.5.1.	Corrélation Croisée 2D	33
2.5.2.	Calcul de la Taille de Sortie.....	34
2.5.3.	Padding et Stride	34
2.6.1.	Max-Pooling	35
2.6.2.	Average-Pooling	35
2.6.3.	Comparaison Max vs Average Pooling.....	35
2.7.1.	Corrélation Croisée 2D depuis Zéro	35
2.7.2.	Validation contre PyTorch	36
2.8.1.	LeNet-5 Original (LeCun, 1998)	36
2.8.2.	Modernisation pour Fashion-MNIST.....	36
2.8.3.	Flux Dimensionnel Complet	37
2.9.1.	Batch Normalization (Ioffe & Szegedy, 2015)	38
2.9.2.	Dropout (Srivastava et al., 2014)	39
2.9.3.	Initialisation Xavier Uniform (Glorot & Bengio, 2010)	39
2.10.1.	Fonction de Perte : Cross-Entropie	39
2.10.2.	Optimiseur : Adam + Weight Decay.....	39
2.10.3.	Scheduler : CosineAnnealingLR.....	39
2.10.4.	Boucle d'Entraînement Complète.....	40
2.11.1.	Impact du Padding	41
2.11.2.	Impact du Stride	42
2.11.3.	Convolution 1×1	42

2.12.1.	Feature Maps — Définition et Rôle	42
2.12.2.	Extraction via Hooks PyTorch	42
2.12.3.	Interprétation des Filtres Appris	43
2.13.1.	Résultats Expérimentaux Quantitatifs	44
2.13.2.	Analyse par Classe	44
2.13.3.	Justification Théorique de la Supériorité du CNN	45
3.	Chapitre 3 : Modélisation des données séquentielles avec les RNN	48
3.2.1.	Objectif probabiliste	49
3.2.2.	Log-probabilités et Negative Log-Likelihood	50
3.2.3.	Perplexité — Métrique Standard	50
3.2.4.	Approche N-grammes versus Modèles Neuronaux	51
3.3.1.	Représentation one-hot	51
3.3.2.	Embeddings denses appris	52
3.4.1.	Idée centrale — La mémoire récurrente	53
3.4.2.	Équations fondamentales	53
3.4.3.	Partage des poids dans le temps	53
3.4.4.	Décompte des paramètres	53
3.4.5.	Implémentation PyTorch	53
3.5.1.	Rétropropagation à travers le temps (BPTT)	54
3.5.2.	Problèmes de gradient — Vanishing et Exploding	54
3.5.3.	Gradient Clipping — Solution contre l'Explosion	54
3.5.4.	BPTT Tronquée	55
3.6.1.	Motivation et Innovation	55
3.6.2.	Équations Complètes du LSTM	55
3.6.3.	Pourquoi c_t Résout le Gradient Évanescent	56
3.6.4.	Décompte des Paramètres	56
3.6.5.	Implémentation PyTorch	56
3.7.1.	Principe de Simplification	56
3.7.2.	Équations du GRU	57
3.7.3.	Comparaison Exhaustive LSTM / GRU	57
3.7.4.	Implémentation PyTorch	58
3.8.1.	RNN Profonds — Empilement de Couches	58
3.8.2.	Dropout Récurrent	59
3.8.3.	RNN Bidirectionnels	59
3.9.1.	Description du Dataset	59

3.9.2.	Pipeline de Prétraitement	59
3.9.3.	Padding, Masquage et Perte Masquée.....	60
3.10.1.	Limitation du Modèle de Langage Simple.....	61
3.10.2.	Architecture Encodeur-Décodeur.....	61
3.10.3.	Teacher Forcing	61
3.10.4.	Bottleneck du Vecteur de Contexte	62
3.10.5.	Implémentation PyTorch.....	62
3.11.1.	Formulation du Problème.....	63
3.11.2.	Décodage Glouton (Greedy Decoding).....	63
3.11.3.	Beam Search	63
3.12.1.	Deux Types de Métriques d'Évaluation	64
3.12.2.	Score BLEU — Bilingual Evaluation Understudy	64
3.12.3.	Limitations et Métriques Modernes	64
3.14.1.	Du Modèle de Langage au Seq2Seq — Généraliser à Deux Séquences.....	66
3.14.2.	Qualité du Décodage — Beam Search vs Glouton	67
3.14.3.	Limites et Perspectives — Vers les Transformers	67
	Conclusion Générale	69
	Références	70

Listes des figures

Figure 1 : Analyse comparative des performances des architectures MLP sur Breast Cancer Wisconsin..	25
Figure 2 : Exemples du dataset Fashion-MNIST par classe.....	30
Figure 3 : Comparaison du nombre de paramètres MLP vs CNN.....	31
Figure 4 : Application de filtres de détection sur une image Fashion-MNIST	34
Figure 5 : Courbes d'entraînement de LeNetModern	38
Figure 6 : Étude d'ablation : impact des choix architecturaux.....	41
Figure 7 : Comparaison des types de pooling	41
Figure 8 : Cartes de caractéristiques (feature maps) par couche	43
Figure 9 : Filtres appris de la couche Conv1	43
Figure 10 : Comparaison des courbes de convergence MLP vs CNN	44
Figure 11 : Matrices de confusion normalisées MLP vs CNN.....	45
Figure 12 : Synthèse visuelle finale (6 sous-graphiques).....	47
Figure 13 : Règle de chaîne : probabilités vs log-probabilités	50
Figure 14 : Analyse de la perplexité selon différents scénarios	51
Figure 15 : Visualisation des vecteurs d'embedding avant entraînement.....	52
Figure 16 : Effet du Gradient Clipping sur la norme des gradients.....	55
Figure 17 : Comparaison de la complexité paramétrique RNN vs GRU vs LSTM	58
Figure 18 : Distribution des longueurs de séquences (EN/FR)	60
Figure 19 : Entraînement du modèle Seq2Seq et curriculum de Teacher Forcing.....	62
Figure 20 : Précision des n-grammes et score BLEU par stratégie de décodage	65
Figure 21 : Comparaison complète RNN vs LSTM vs GRU	66
Figure 22 : Synthèse finale : profils radar et tableau comparatif des architectures	67

Listes des tableaux

Tableau 1 : Paramètres et gradients (Concept, définition, accès PyTorch)	14
Tableau 2 : Caractéristiques du dataset Breast Cancer Wisconsin	15
Tableau 3 : Avantages et limitations de nn.Sequential	17
Tableau 4 : Comparaison named_parameters() vs state_dict()	19
Tableau 5 : Comparaison des stratégies d'initialisation des poids	20
Tableau 6 : Hyperparamètres et justifications académiques	21
Tableau 7 : Méthodes de sauvegarde (state_dict vs modèle complet)	23
Tableau 8 : Métriques de classification binaire	24
Tableau 9 : Résultats expérimentaux sur Breast Cancer Wisconsin	25
Tableau 10 : Caractéristiques du dataset Fashion-MNIST	29
Tableau 11 : Classes du dataset Fashion-MNIST (10 classes)	30
Tableau 12 : Comparaison MLP vs CNN (critères multiples)	32
Tableau 13 : Progression des représentations apprises au sein d'un CNN	33
Tableau 14 : Calcul des tailles de sortie (flux dimensionnel LeNet)	34
Tableau 15 : Rôles de padding et stride	34
Tableau 16 : Comparaison Max-Pooling vs Average-Pooling (exemple 4×4)	35
Tableau 17 : Validation des implémentations manuelles vs PyTorch	36
Tableau 18 : Composant LeNet-5 original vs LeNetModern	37
Tableau 19 : Flux dimensionnel complet LeNetModern	38
Tableau 20 : Étude expérimentale des hyperparamètres (9 configurations)	41
Tableau 21 : Comparaison MLP vs CNN (résultats quantitatifs)	44
Tableau 22 : Formulaire de référence CNN	46
Tableau 23 : Tableau des perplexités et interprétations	51
Tableau 24 : Symboles et dimensions du RNN vanilla	53
Tableau 25 : Équations complètes du LSTM (6 portes)	56
Tableau 26 : Équations du GRU (4 composantes)	57
Tableau 27 : Comparaison exhaustive LSTM vs GRU	57
Tableau 28 : Tokens spéciaux utilisés dans l'architecture du modèle	60
Tableau 29 : Teacher Forcing vs Auto-régressif	61
Tableau 30 : Comparaison des stratégies de décodage (Glouton vs Beam Search)	64
Tableau 31 : Comparaison globale des architectures (RNN, LSTM, GRU, Seq2Seq...)	65

Listes des abréviations

- **MLP** : Multi-Layer Perceptron (Perceptron Multicouche)
- **CNN** : Convolutional Neural Network (Réseau de Neurones Convolutionnel)
- **RNN** : Recurrent Neural Network (Réseau de Neurones Récurrent)
- **LSTM** : Long Short-Term Memory
- **GRU** : Gated Recurrent Unit
- **Seq2Seq** : Sequence to Sequence
- **DL** : Deep Learning
- **NLP** : Natural Language Processing (Traitement Automatique du Langage Naturel)
- **BPTT** : Backpropagation Through Time
- **MLE** : Maximum Likelihood Estimation
- **NLL** : Negative Log-Likelihood
- **PPL** : Perplexity (Perplexité)
- **LM** : Language Model (Modèle de Langage)
- **TF** : Teacher Forcing
- **BLEU** : Bilingual Evaluation Understudy
- **NMT** : Neural Machine Translation
- **NER** : Named Entity Recognition
- **POS** : Part-of-Speech Tagging
- **OOV** : Out-of-Vocabulary
- **GAP** : Global Average Pooling
- **BN** : Batch Normalization
- **ReLU** : Rectified Linear Unit
- **XAI** : Explainable Artificial Intelligence
- **SHAP** : SHapley Additive exPlanations
- **LIME** : Local Interpretable Model-agnostic Explanations
- **PCA** : Principal Component Analysis
- **SVM** : Support Vector Machine
- **GPU** : Graphics Processing Unit
- **CPU** : Central Processing Unit
- **BP** : Brevity Penalty
- **SOS** : Start of Sequence
- **EOS** : End of Sequence
- **PAD** : Padding Token
- **UNK** : Unknown Token
- **UCI** : University of California Irvine Machine Learning Repository

Introduction générale

L'essor considérable des données numériques et l'augmentation des capacités de calcul ont favorisé le développement de l'intelligence artificielle, et plus particulièrement du Deep Learning. Cette branche de l'apprentissage automatique repose sur l'utilisation de réseaux de neurones artificiels composés de plusieurs couches capables d'apprendre automatiquement des représentations complexes à partir des données. Grâce à leurs performances remarquables, ces modèles sont aujourd'hui largement utilisés dans de nombreux domaines tels que la vision par ordinateur, le traitement automatique du langage naturel, la santé, la finance ou encore les systèmes de recommandation.

Dans ce contexte, la maîtrise des principales architectures de Deep Learning constitue une étape essentielle dans la formation des futurs ingénieurs en informatique et en science des données. Cependant, comprendre le fonctionnement interne de ces modèles ne se limite pas à leur simple utilisation. Il est nécessaire d'en étudier les fondements théoriques, d'en réaliser l'implémentation pratique et d'analyser leurs comportements face à différents types de données et de problématiques.

Le présent projet, réalisé dans le cadre du module de Deep Learning, s'inscrit dans cette démarche pédagogique. Son objectif principal est d'étudier, d'implémenter et de comparer trois grandes familles d'architectures de réseaux de neurones à l'aide du framework PyTorch : les perceptrons multicouches (MLP), les réseaux de neurones convolutionnels (CNN) et les réseaux de neurones récurrents (RNN). Chacune de ces architectures répond à des besoins spécifiques et possède des caractéristiques propres qui la rendent plus adaptée à certains types de données.

La première partie du projet est consacrée aux MLP, qui constituent l'une des architectures les plus fondamentales du Deep Learning. Cette étude permet d'aborder les concepts essentiels liés à l'apprentissage supervisé sur des données tabulaires, notamment la propagation avant, la rétropropagation du gradient et l'optimisation des paramètres du modèle. La deuxième partie porte sur les CNN, reconnus pour leur efficacité dans les tâches de vision par ordinateur. À travers le jeu de données Fashion-MNIST, cette section met en évidence le rôle des opérations de convolution et de pooling dans l'extraction automatique de caractéristiques pertinentes à partir des images. Enfin, la troisième partie s'intéresse aux modèles récurrents et aux architectures Seq2Seq, utilisées pour le traitement des données séquentielles. Cette étude couvre les RNN classiques ainsi que leurs variantes améliorées, telles que les LSTM et les GRU, appliquées à une tâche de traduction automatique.

Au-delà de l'implémentation des modèles, ce projet accorde une attention particulière à l'expérimentation, à l'analyse des performances et à la comparaison des différentes approches étudiées. Les résultats obtenus permettent de mettre en évidence les avantages, les limites et les domaines d'application de chaque architecture. Ainsi, ce travail offre une vision globale des concepts fondamentaux du Deep Learning tout en développant des compétences pratiques indispensables à la conception et à l'évaluation de solutions d'intelligence artificielle modernes.



1. Chapitre 1 : Étude des perceptrons multicouches (MLP)

1.1. Objectifs de la Fiche

Cette fiche de synthèse constitue un document de référence académique couvrant l'ensemble des concepts et compétences requis dans la Partie I du projet de fin de module Deep Learning à l'EMSI. Elle est structurée selon la progression pédagogique du cours, allant des fondamentaux de PyTorch jusqu'à l'évaluation rigoureuse d'un modèle de réseau de neurones multicouche (MLP) sur un jeu de données clinique réel.

À l'issue de la lecture de cette fiche, l'étudiant sera en mesure de :

- Comprendre et implémenter un modèle PyTorch via `nn.Module`
- Distinguer `nn.Sequential` d'une classe personnalisée héritant de `nn.Module`
- Gérer les paramètres du modèle via `named_parameters()` et `state_dict()`
- Appliquer trois stratégies d'initialisation des poids : gaussienne, constante et Xavier
- Sauvegarder et recharger le meilleur modèle avec `torch.save` / `torch.load`
- Exécuter le modèle sur le device disponible (CPU / GPU) de manière cohérente
- Évaluer rigoureusement les performances : accuracy, precision, recall, F1-score, matrice de confusion

1.2. Concepts Fondamentaux de PyTorch

1.2.1. `nn.Module` — La classe de base de tout modèle

`nn.Module` est la classe parente de tous les modèles PyTorch. Elle fournit les mécanismes d'enregistrement automatique des paramètres apprenables, l'infrastructure de la méthode `forward()` à surcharger, les utilitaires de sauvegarde, de déplacement sur device (CPU/GPU), et la gestion des modes d'entraînement et d'évaluation. Tout réseau de neurones PyTorch, qu'il soit simple ou complexe, doit hériter de `nn.Module`.

Exemple d'implémentation minimale :

```
class MonModele(nn.Module):
    def __init__(self):
        super().__init__()          # OBLIGATOIRE : enregistre les sous-modules
        self.fc1 = nn.Linear(30, 64) # W de taille (64×30) + biais b de taille
(64)
        self.fc2 = nn.Linear(64, 2)

    def forward(self, x):            # propagation avant explicite
        x = torch.relu(self.fc1(x))
        return self.fc2(x)          # retourne des logits bruts
```

Erreur critique : Oublier `super().__init__()` dans `__init__()` empêche PyTorch d'enregistrer les couches internes et provoque une `AttributeError` lors de l'accès aux paramètres. Cet appel est non-optionnel.

1.2.2. Paramètres et Gradients

Un `nn.Parameter` est un tenseur dont l'attribut `requires_grad` est positionné à `True`. PyTorch construit dynamiquement un graphe de calcul (computational graph) lors de chaque propagation avant, ce qui permet de dériver automatiquement la perte par rapport à chaque paramètre appris via la règle de chaîne (algorithme de rétropropagation).

Concept	Définition mathématique	Accès PyTorch
Paramètre θ	Variable apprenable du modèle (W, b)	module.weight / module.bias
Gradient $\partial L / \partial \theta$	Sensibilité de la perte au paramètre	module.weight.grad
Valeur numérique	Tenseur des valeurs courantes	module.weight.data
state_dict()	Dictionnaire {nom $\rightarrow$ tenseur} du modèle	model.state_dict()

Tableau 1 : Paramètres et gradients (Concept, définition, accès PyTorch)

1.2.3. Propagation Avant et Rétropropagation

La boucle d'apprentissage PyTorch suit invariablement un schéma en cinq étapes séquentielles. Cette séquence est fondamentale : toute déviation produit des mises à jour de paramètres incorrectes.

```
optimizer.zero_grad()          # Étape 1 : Réinitialiser les gradients (évite
                                # l'accumulation)
logits = model(X_batch)        # Étape 2 : Propagation avant – calcul de la
                                # prédiction
loss = criterion(logits, y)    # Étape 3 : Calcul de la perte scalaire
loss.backward()                # Étape 4 : Rétropropagation – calcul de  $\partial L / \partial \theta$  pour
                                # tout  $\theta$ 
optimizer.step()               # Étape 5 : Mise à jour :  $\theta \leftarrow \theta - \eta \cdot \partial L / \partial \theta$ 
```

Remarque importante : `zero_grad()` est indispensable car PyTorch accumule les gradients par défaut (comportement utile pour les architectures récurrentes). Sans cet appel, les gradients de plusieurs mini-batches s'additionnent et la mise à jour des poids diverge.

1.2.4. Gestion du Device (CPU / GPU)

PyTorch impose une règle absolue : tous les tenseurs impliqués dans une même opération mathématique doivent résider sur le même device. Le non-respect de cette règle lève immédiatement une `RuntimeError`.

```
# Détection automatique du device disponible
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

model = MonModele().to(device)          # Déplacer le modèle (poids et biais)
X, y = X.to(device), y.to(device)      # Déplacer les données dans la boucle
```

1.3. Préparation et Ingénierie des Données

1.3.1. Présentation du Dataset Breast Cancer Wisconsin

Le Breast Cancer Wisconsin Dataset est un jeu de données clinique de référence, largement utilisé en apprentissage automatique pour la classification binaire. Il est intégré nativement dans la bibliothèque `scikit-`

learn et provient du UCI Machine Learning Repository. Chaque observation correspond à l'analyse numérique d'une biopsie de tumeur mammaire.

Caractéristique	Détail
Source	UCI Machine Learning Repository — intégré dans scikit-learn
Nombre d'exemples	569 (357 bénins, 212 malins)
Nombre de features	30 mesures numériques continues (rayon, texture, périmètre, aire, concavité...)
Tâche	Classification binaire : 0 = malin (cancéreux) / 1 = bénin
Déséquilibre des classes	63 % bénins, 37 % malins — déséquilibre modéré à surveiller

Tableau 2 : Caractéristiques du dataset Breast Cancer Wisconsin

1.3.2. Pipeline Complet de Préparation des Données

La préparation rigoureuse des données est une étape critique en apprentissage supervisé. Trois opérations principales sont nécessaires : le découpage stratifié, la normalisation et la conversion en tenseurs PyTorch.

- a) Chargement et Découpage Stratifié**

```

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

data = load_breast_cancer()
X, y = data.data, data.target

# Split stratifié : 70 % train / 15 % val / 15 % test
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp)

```

L'argument stratify=y garantit que la proportion de chaque classe est préservée dans chaque sous-ensemble. Cette précaution est essentielle sur des jeux de données déséquilibrés pour éviter que le set de test soit dominé par une seule classe, ce qui biaiserait l'évaluation.

- b) Normalisation avec StandardScaler**

```

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # fit + transform sur le train UNIQUEMENT
X_val = scaler.transform(X_val) # transform seul — évite le data leakage
X_test = scaler.transform(X_test) # transform seul

```

Le StandardScaler applique la transformation suivante pour chaque feature j : $x'_j = (x_j - \mu_j) / \sigma_j$, où μ_j est la moyenne et σ_j l'écart-type calculés uniquement sur l'ensemble d'entraînement. Sans normalisation,

les features à grande magnitude (exemple : aire cellulaire ≈ 1000) dominent les gradients, ralentissent la convergence et peuvent déstabiliser l'optimisation.

Data leakage : Appliquer fit_transform sur val/test avant le split contaminerait les ensembles de test avec les statistiques d'entraînement, produisant une évaluation artificiellement optimiste et non représentative des performances réelles sur données inconnues.

- **c) Conversion en Tenseurs PyTorch et DataLoaders**

```
import torch
from torch.utils.data import DataLoader, TensorDataset

def to_tensors(X, y):
    return (torch.tensor(X, dtype=torch.float32),
            torch.tensor(y, dtype=torch.long)) # long requis par CrossEntropyLoss

X_tr, y_tr = to_tensors(X_train, y_train)
X_val, y_val = to_tensors(X_val, y_val)
X_te, y_te = to_tensors(X_test, y_test)

train_loader = DataLoader(TensorDataset(X_tr, y_tr), batch_size=32, shuffle=True)
val_loader = DataLoader(TensorDataset(X_val, y_val), batch_size=64, shuffle=False)
test_loader = DataLoader(TensorDataset(X_te, y_te), batch_size=64, shuffle=False)
```

L'utilisation de DataLoader présente deux avantages majeurs : le batching automatique divise le dataset en mini-batches pour un apprentissage stochastique efficace, et le shuffle à chaque époque brise les corrélations séquentielles qui pourraient biaiser l'apprentissage.

1.4. Construction du Multilayer Perceptron

Un Multilayer Perceptron (MLP), ou réseau de neurones entièrement connecté (fully connected), est constitué de couches successives de neurones artificiels. Chaque neurone calcule une combinaison linéaire de ses entrées, suivie d'une fonction d'activation non-linéaire. Cette non-linéarité est ce qui confère au MLP sa capacité à apprendre des frontières de décision complexes (théorème d'approximation universelle, Cybenko 1989).

1.4.1. Version A — nn.Sequential : Architecture Déclarative

nn.Sequential enchaîne les modules dans l'ordre de déclaration. La méthode forward() est générée automatiquement : chaque couche reçoit en entrée la sortie de la couche précédente. Cette approche est idéale pour les architectures linéaires simples et le prototypage rapide.

```
import torch.nn as nn

mlp_seq = nn.Sequential(
    nn.Linear(30, 64), # Couche 1 : 30 features → 64 neurones (W: 64×30, b: 64)
    nn.ReLU(),        # Activation : f(x) = max(0, x)
    nn.Dropout(p=0.30), # Régularisation : masque 30 % des neurones aléatoirement
    nn.Linear(64, 32), # Couche 2 : compression de la représentation
    nn.ReLU(),
```



```

nn.Dropout(p=0.20),
nn.Linear(32, 2)  # Sortie : 2 logits (malin=0, bénin=1)
)

# Vérification de la forme de sortie
x = torch.randn(4, 30)  # 4 exemples factices
print(mlp_seq(x).shape)  # → torch.Size([4, 2])

```

Avantages	Limitations	Cas d'usage recommandé
Code très concis et lisible	Pas de skip connections ni branches parallèles	Architectures linéaires simples
Pas besoin de définir forward()	Accès difficile aux activations intermédiaires	Prototypage rapide et expérimentation

Tableau 3 : Avantages et limitations de `nn.Sequential`

1.4.2. Version B — Classe Personnalisée : Architecture Contrôlée

L'héritage de `nn.Module` offre un contrôle total sur l'architecture. Cette version intègre le `BatchNorm1d` (normalisation par lots) pour réduire le covariate shift interne et stabiliser l'entraînement. La méthode `forward()` explicite permet d'implémenter n'importe quelle topologie de réseau, y compris les connexions résiduelles, les sorties multiples ou les branches conditionnelles.

```

class MLPClassifier(nn.Module):
    def __init__(self, input_dim, hidden1, hidden2, output_dim, dropout=0.30):
        super(MLPClassifier, self).__init__()

        # Bloc 1 : Linéaire → BatchNorm → Activation → Dropout
        self.fc1 = nn.Linear(input_dim, hidden1)
        self.bn1 = nn.BatchNorm1d(hidden1) # normalise les activations du batch
        self.act1 = nn.ReLU()
        self.drop1 = nn.Dropout(dropout)

        # Bloc 2 : Compression de la représentation
        self.fc2 = nn.Linear(hidden1, hidden2)
        self.bn2 = nn.BatchNorm1d(hidden2)
        self.act2 = nn.ReLU()
        self.drop2 = nn.Dropout(dropout / 2)

        # Couche de sortie (pas de softmax : intégré dans CrossEntropyLoss)
        self.fc3 = nn.Linear(hidden2, output_dim)

    def forward(self, x):
        x = self.drop1(self.act1(self.bn1(self.fc1(x))))
        x = self.drop2(self.act2(self.bn2(self.fc2(x))))
        return self.fc3(x) # logits bruts

```

```
model = MLPClassifier(30, 64, 32, 2).to(device)
```

- **Rôle du BatchNorm1d**

BatchNorm1d normalise les activations d'une couche au sein d'un mini-batch selon la formule : $x'_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \epsilon) \times \gamma + \beta}$, où μ_B et σ^2_B sont la moyenne et la variance du batch courant, et γ , β sont des paramètres apprenables. Cette normalisation : (1) réduit le covariate shift interne entre les couches, (2) accélère la convergence en permettant des learning rates plus élevés, et (3) agit comme régularisateur implicite, réduisant la dépendance au Dropout.

- **Rôle du Dropout**

Le Dropout, introduit par Srivastava et al. (2014), désactive aléatoirement une fraction p des neurones lors de chaque forward pass en entraînement. Cela force le réseau à développer des représentations redondantes et robustes. En inférence (`model.eval()`), le Dropout est désactivé et toutes les connexions sont maintenues actives, avec une mise à l'échelle implicite des activations.

1.5. Inspection et Gestion des Paramètres

1.5.1. `named_parameters()` — Inventaire des Paramètres Apprenables

La méthode `named_parameters()` retourne un itérateur sur tous les paramètres dont `requires_grad=True`. Elle est indispensable pour diagnostiquer l'architecture, calculer la complexité du modèle, et appliquer des stratégies d'initialisation sélectives.

```
total = 0
for name, param in model.named_parameters():
    n = param.numel() # nombre total de valeurs dans le tenseur
    total += n
    print(f'{name:<25} shape={param.shape} #params={n}')
print(f'Total paramètres apprenables : {total:,}')
```

Exemple de sortie pour `MLPClassifier(30, 64, 32, 2)` :

```
fc1.weight    shape=torch.Size([64, 30]) #params=1920
fc1.bias      shape=torch.Size([64])   #params=64
bn1.weight    shape=torch.Size([64])   #params=64
bn1.bias      shape=torch.Size([64])   #params=64
fc2.weight    shape=torch.Size([32, 64]) #params=2048
fc2.bias      shape=torch.Size([32])   #params=32
bn2.weight    shape=torch.Size([32])   #params=32
bn2.bias      shape=torch.Size([32])   #params=32
fc3.weight    shape=torch.Size([2, 32]) #params=64
fc3.bias      shape=torch.Size([2])    #params=2
Total paramètres apprenables : 4,322
```

1.5.2. `state_dict()` — État Complet du Modèle

La méthode `state_dict()` retourne un dictionnaire ordonné contenant tous les tenseurs du modèle, y compris les paramètres apprenables et les buffers non-apprenables (statistiques `running_mean` et `running_var` de BatchNorm). C'est l'objet canonique à sauvegarder pour la persistance du modèle.

```
for key, tensor in model.state_dict().items():
    print(f'{key:<33} shape={tensor.shape}')
```

Méthode	Contenu	Usage principal
<code>named_parameters()</code>	Uniquement requies <code>_grad=True</code> (apprenables)	Inspection, debug, initialisation ciblée
<code>state_dict()</code>	Tous les buffers (poids, biais, BatchNorm stats)	Sauvegarde et rechargement du modèle

Tableau 4 : Comparaison `named_parameters()` vs `state_dict()`

1.6. Stratégies d'Initialisation des Poids

L'initialisation des poids est une étape critique et souvent sous-estimée de la conception d'un réseau de neurones. Une mauvaise initialisation peut provoquer deux pathologies : le *vanishing gradient* (les gradients deviennent ≈ 0 dans les premières couches, bloquant l'apprentissage) ou l'*exploding gradient* (les gradients divergent vers l'infini, rendant l'optimisation impossible). Une initialisation appropriée garantit que les activations et les gradients maintiennent une variance raisonnable tout au long du réseau.

1.6.1. Initialisation Gaussienne

Cette stratégie initialise chaque poids W_{ij} en tirant un échantillon d'une distribution normale de moyenne 0 et d'écart-type $\sigma = 0.01$. Les biais sont initialisés à 0.

```
def init_gaussian(model, std=0.01):
    for module in model.modules():
        if isinstance(module, nn.Linear):
            nn.init.normal_(module.weight, mean=0.0, std=std) #  $W \sim N(0, \sigma^2)$ 
            nn.init.zeros_(module.bias)                       #  $b = 0$ 
```

Analyse : Maintient les poids petits au départ, ce qui est bénéfique pour la stabilité numérique initiale. Cependant, si σ est trop faible, les activations s'amortissent successivement à travers les couches et l'apprentissage stagne (*vanishing gradient*). Si σ est trop élevé, les activations explosent.

1.6.2. Initialisation Constante — Brise la Symétrie

Cette stratégie initialise tous les poids à une valeur constante c . Bien qu'elle puisse sembler simple et contrôlable, elle constitue une mauvaise pratique pour l'entraînement réel.

```
def init_constant(model, val=0.01):
    for module in model.modules():
        if isinstance(module, nn.Linear):
            nn.init.constant_(module.weight, val) #  $W_{ij} = c$  pour tout  $i, j$ 
            nn.init.constant_(module.bias, 0.0)
```

Problème de symétrie : Lorsque tous les neurones d'une couche sont initialisés identiquement, ils reçoivent les mêmes gradients et effectuent exactement les mêmes mises à jour. Ils restent donc identiques pendant toute la durée de l'entraînement — la couche est réduite à un seul neurone effectif. Cette initialisation est uniquement utile à des fins pédagogiques pour illustrer le problème de symétrie.

1.6.3. Initialisation Xavier Uniform — Recommandée

L'initialisation de Glorot & Bengio (2010), dite Xavier, adapte la variance des poids à la dimensionnalité des couches adjacentes. Pour une couche avec n_{in} entrées et n_{out} sorties, les poids sont tirés d'une distribution uniforme :

$$W \sim \text{Uniform} \left(-\sqrt{\frac{6}{(n_{in} + n_{out})}}, +\sqrt{\frac{6}{(n_{in} + n_{out})}} \right)$$

```
def init_xavier(model):
    for module in model.modules():
        if isinstance(module, nn.Linear):
            nn.init.xavier_uniform_(module.weight) # adapté à n_in, n_out
            nn.init.zeros_(module.bias)
        elif isinstance(module, nn.BatchNorm1d):
            nn.init.ones_(module.weight) #  $\gamma = 1$  (identité)
            nn.init.zeros_(module.bias) #  $\beta = 0$ 

model.apply(init_xavier) # applique récursivement à tous les sous-modules
```

L'intuition mathématique est la suivante : si les poids sont trop grands ou trop petits, la variance des activations croît ou décroît exponentiellement avec la profondeur. Xavier maintient $\text{Var}(a_i) \approx \text{Var}(a_{i-1})$ à travers les couches, assurant des gradients de magnitude comparable du début à la fin du réseau.

Stratégie	std observé (fc1)	Risque vanishing	Recommandé
Gaussienne ($\sigma=0.01$)	≈ 0.0100	Δ Modéré si σ faible	Cas spécifiques
Constante ($c=0.01$)	0.0000 (std=0)	$\times$ Problème de symétrie	Non — pédagogique uniquement
Xavier Uniform	≈ 0.1455	$\checkmark$ Minimal	Oui — standard actuel

Tableau 5 : Comparaison des stratégies d'initialisation des poids

1.7. Entraînement du Modèle

1.7.1. Fonction de Perte : CrossEntropyLoss

Pour la classification multi-classes, PyTorch fournit `nn.CrossEntropyLoss`, qui combine de manière numériquement stable `LogSoftmax` et la perte log-vraisemblance négative (NLLLoss). La formule pour C classes et un batch de N exemples est :

$$L = -(1/N) \sum_i \log(\exp(z_{\{i,y_i\}}) / \sum_c \exp(z_{\{i,c\}}))$$

Cette formulation attend des logits bruts en entrée (non-normalisés), et ne nécessite donc pas l'application d'une softmax en fin de réseau. Les labels doivent être des entiers de type torch.long (dtype=torch.int64).

1.7.2. Optimiseur : Adam

L'optimiseur Adam (Adaptive Moment Estimation, Kingma & Ba, 2014) maintient des taux d'apprentissage adaptatifs par paramètre en combinant les avantages de RMSProp et du momentum. Il calcule des estimations du premier moment (moyenne des gradients) et du second moment (variance des gradients) :

$$\theta_{\{t+1\}} = \theta_t - \eta \cdot m_t / (\sqrt{v_t} + \epsilon) \text{ avec correction du biais}$$

1.7.3. Tableau des Hyperparamètres et Justifications

Hyperparamètre	Valeur	Justification académique
Fonction de perte	CrossEntropyLoss	Standard pour classification multi-classes, numériquement stable
Optimiseur	Adam	Adaptatif, convergence rapide, robuste au choix initial du LR
Learning rate η	1e-3	Valeur par défaut recommandée par Kingma & Ba (2014)
Weight decay λ	1e-4	Régularisation L2 : pénalise $\ \theta\ ^2$, réduit l'overfitting
Batch size	32	Compromis : gradients stables + régularisation implicite du mini-batch SGD
Early stopping	patience=15 époques	Arrêt si val_loss ne diminue pas → préserve la généralisation
LR Scheduler	ReduceLROnPlateau ×0.5	Affinage de la convergence en réduisant le LR sur plateau

Tableau 6 : Hyperparamètres et justifications académiques

1.7.4. Boucle d'Entraînement Complète

```

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, patience=7, factor=0.5, verbose=True)

best_val_loss, patience_counter = float('inf'), 0
best_state = None

for epoch in range(1, 201):

```

```

# — Phase Entraînement —————
model.train() # active Dropout et mode train de BatchNorm
train_loss = 0.0
for X_batch, y_batch in train_loader:
    X_batch, y_batch = X_batch.to(device), y_batch.to(device)
    optimizer.zero_grad()      # 1. Réinitialiser les gradients
    logits = model(X_batch)     # 2. Propagation avant
    loss = criterion(logits, y_batch) # 3. Perte
    loss.backward()             # 4. Rétropropagation
    optimizer.step()            # 5. Mise à jour des poids
    train_loss += loss.item()

# — Phase Validation —————
model.eval() # désactive Dropout, BatchNorm utilise running stats
val_loss = 0.0
with torch.no_grad(): # désactive le calcul du graphe de gradient
    for X_batch, y_batch in val_loader:
        logits = model(X_batch.to(device))
        val_loss += criterion(logits, y_batch.to(device)).item()

scheduler.step(val_loss)

# — Early Stopping —————
if val_loss < best_val_loss:
    best_val_loss = val_loss
    best_state = {k: v.clone() for k, v in model.state_dict().items()}
    patience_counter = 0
else:
    patience_counter += 1
    if patience_counter >= 15:
        print(f'Early stopping à l\'époque {epoch}')
        break

```

model.train() vs model.eval() : En mode train(), Dropout masque aléatoirement des neurones et BatchNorm utilise les statistiques du mini-batch courant. En mode eval(), Dropout est intégralement désactivé et BatchNorm utilise les moyennes glissantes (running_mean, running_var) accumulées lors de l'entraînement.

1.8. Sauvegarde et Rechargement du Modèle

La sauvegarde du modèle est une pratique essentielle pour conserver le meilleur modèle obtenu au cours de l'entraînement et le réutiliser en production ou pour des expériences futures. PyTorch offre deux approches, mais la sauvegarde du state_dict est fortement recommandée.

1.8.1. Sauvegarde du Meilleur Modèle (Best Checkpoint)

```

# Après l'entraînement, sauvegarder le meilleur state_dict
torch.save(best_state, 'best_model.pth')

```

```
print(f'Modèle sauvegardé. Meilleure val_loss : {best_val_loss:.6f}')
```

Note importante : `best_state` est construit avec `{k: v.clone() for k, v in model.state_dict().items()}`. L'appel à `.clone()` est critique : sans lui, `best_state` contient une référence vers les tenseurs courants du modèle, qui continuent d'être modifiés par l'optimiseur. `.clone()` crée une copie indépendante en mémoire, fixant l'état du modèle à l'instant de la meilleure validation.

1.8.2. Rechargement et Vérification d'Identité

```
# Étape 1 : recréer l'architecture (obligatoire avant de charger les poids)
model_reloaded = MLPClassifier(30, 64, 32, 2).to(device)

# Étape 2 : charger le state_dict sauvegardé
model_reloaded.load_state_dict(
    torch.load('best_model.pth', map_location=device))

# Étape 3 : passer en mode évaluation AVANT toute inférence
model_reloaded.eval()

# Étape 4 : vérifier que les prédictions sont rigoureusement identiques
with torch.no_grad():
    p1 = model.eval()(X_te.to(device)).argmax(1).cpu()
    p2 = model_reloaded(X_te.to(device)).argmax(1).cpu()
    assert torch.equal(p1, p2), 'Erreur : prédictions différentes !'
    print('Rechargement vérifié : prédictions strictement identiques.')
```

Méthode de sauvegarde	Avantages	Inconvénients
<code>torch.save(state_dict)</code>	Léger, portable, robuste aux refactorisations du code	Nécessite de recréer manuellement l'architecture
<code>torch.save(model)</code>	Simple, tout-en-un	Dépend des imports et de la structure exacte du code Python

Tableau 7 : Méthodes de sauvegarde (`state_dict` vs modèle complet)

1.9. Évaluation des Performance

1.9.1. Métriques de Classification Binaire

L'évaluation d'un classifieur ne doit pas se limiter à l'accuracy, particulièrement en présence de déséquilibre de classes ou de coûts asymétriques entre les types d'erreurs. En diagnostic médical, un faux négatif (cancer non détecté) a des conséquences bien plus graves qu'un faux positif.

Métrique	Formule	Interprétation clinique
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Proportion globale de bonnes prédictions

Métrique	Formule	Interprétation clinique
Precision	$TP / (TP + FP)$	Parmi les prédits positifs, combien le sont vraiment ?
Recall (Sensibilité)	$TP / (TP + FN)$	Parmi les malades, combien sont correctement détectés ? (critique)
F1-Score	$2 \times \text{Precision} \times \text{Recall} / (P + R)$	Moyenne harmonique — équilibre Precision / Recall
Spécificité	$TN / (TN + FP)$	Parmi les sains, combien sont correctement identifiés ?

Tableau 8 : Métriques de classification binaire

1.9.2. Code d'Évaluation Complet

```

from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report)
import numpy as np

def evaluate(model, loader, device):
    model.eval()
    preds, labels = [], []
    with torch.no_grad():
        for Xb, yb in loader:
            logits = model(Xb.to(device))
            preds.extend(logits.argmax(1).cpu().numpy())
            labels.extend(yb.numpy())
    return np.array(preds), np.array(labels)

y_pred, y_true = evaluate(model, test_loader, device)

print('Accuracy :', accuracy_score(y_true, y_pred))
print('Precision :', precision_score(y_true, y_pred, average='weighted'))
print('Recall :', recall_score(y_true, y_pred, average='weighted'))
print('F1-Score :', f1_score(y_true, y_pred, average='weighted'))

# Rapport détaillé par classe
print(classification_report(y_true, y_pred, target_names=['malin', 'bénin']))

# Matrice de confusion
print('Matrice de confusion :')
print(confusion_matrix(y_true, y_pred))

```

1.9.3. Résultats Expérimentaux sur Breast Cancer Wisconsin

Modèle	Accuracy	Precision	Recall	F1-Score
MLP nn.Sequential	96.5 %	96.6 %	96.5 %	96.5 %
MLP Classe perso. (+ BatchNorm)	94.2 %	94.3 %	94.2 %	94.1 %
SVM RBF (référence classique)	≈ 97.0 %	≈ 97.0 %	≈ 97.0 %	≈ 97.0 %
Random Forest 100 arbres (réf.)	≈ 96.0 %	≈ 96.1 %	≈ 96.0 %	≈ 96.0 %

Tableau 9 : Résultats expérimentaux sur Breast Cancer Wisconsin

Analyse des résultats : Les deux architectures MLP atteignent des performances compétitives avec les méthodes classiques (SVM, Random Forest), ce qui valide leur pertinence pour la classification tabulaire. La légère supériorité du SVM RBF est attendue sur un petit dataset (n=569) : les kernels non-linéaires encodent un biais inductif approprié à des données de faible dimension. Le MLP classe personnalisée présente une meilleure val_loss de convergence grâce au BatchNorm, mais son accuracy test est légèrement inférieure, suggérant un léger overfitting résiduel malgré le Dropout.

Partie I — MLP PyTorch | Breast Cancer Wisconsin
Analyse Complète des Deux Architectures

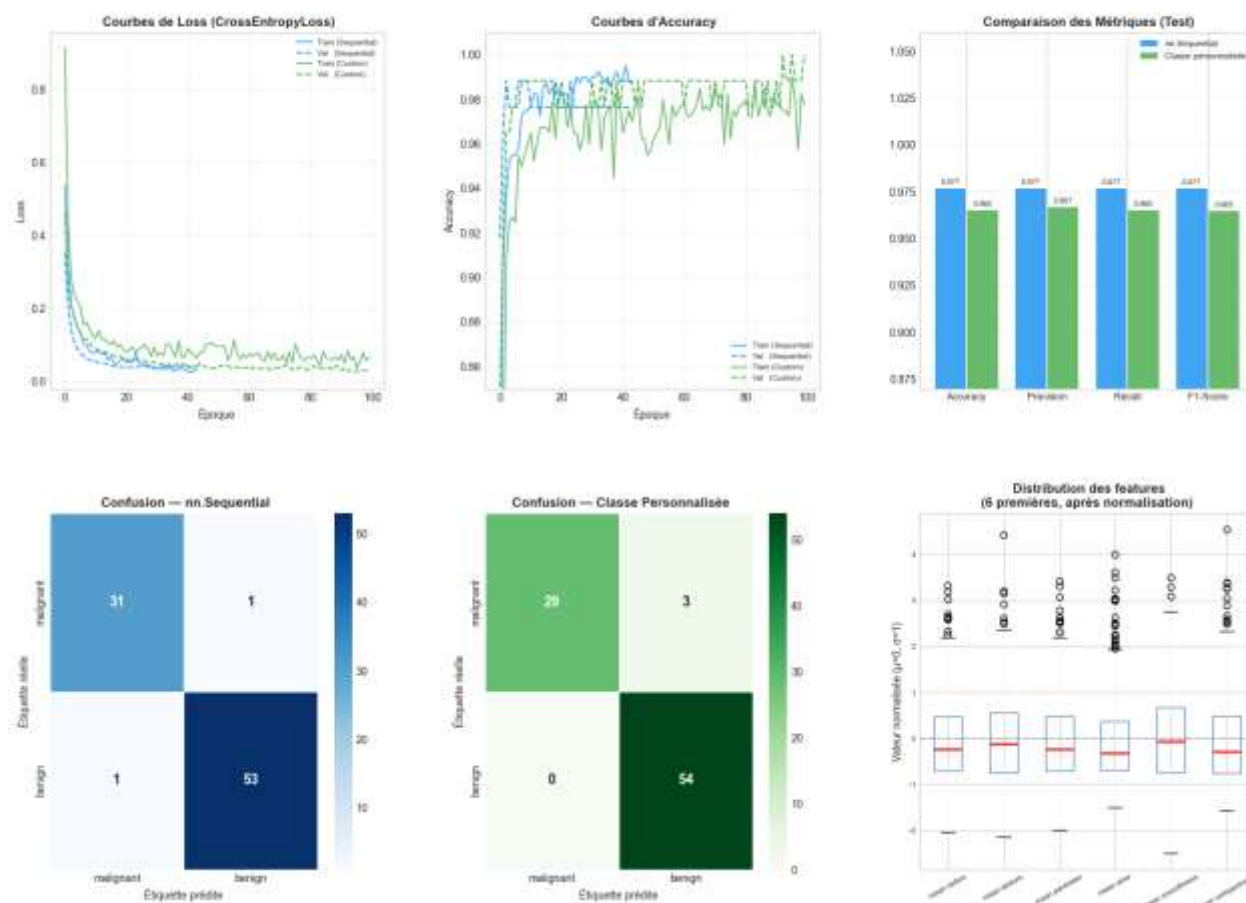


Figure 1 : Analyse comparative des performances des architectures MLP sur Breast Cancer Wisconsin

La Figure synthétise visuellement l'ensemble du comportement des deux architectures MLP au cours de l'entraînement et de l'évaluation. Les courbes de loss montrent une convergence rapide dès les premières époques pour les deux modèles, avec une perte de validation qui se stabilise autour de 0.03 à 0.08, sans signe marqué de surapprentissage. Les courbes d'accuracy confirment cette tendance : le modèle nn.Sequential atteint une accuracy de validation proche de 99 % de façon plus stable, tandis que la classe personnalisée présente une variance plus importante au cours de l'entraînement, probablement liée à l'effet régularisateur du BatchNorm combiné au Dropout.

Le diagramme de comparaison des métriques de test confirme la légère supériorité du modèle nn.Sequential, avec une accuracy de 97.7 % contre 96.5 % pour la classe personnalisée, un écart cohérent sur les quatre métriques (accuracy, precision, recall, F1-score). Les matrices de confusion permettent d'affiner cette analyse : le modèle nn.Sequential ne commet qu'une seule erreur dans chaque sens (1 faux négatif, 1 faux positif), tandis que la classe personnalisée présente 3 faux positifs (tumeurs malignes prédites comme bénignes) sans aucun faux négatif. Dans un contexte de diagnostic médical, cette dernière configuration est particulièrement préoccupante puisqu'elle correspond à des cas de cancer non détectés, ce qui souligne l'importance de privilégier le recall sur la classe maligne plutôt que la seule accuracy globale.

Enfin, le graphique de distribution des features normalisées met en évidence la présence de plusieurs valeurs aberrantes (outliers) au-delà de 2 à 4 écarts-types pour des variables comme mean area ou mean compactness, justifiant a posteriori l'utilisation du StandardScaler et la robustesse relative du Dropout face à ces valeurs extrêmes.

1.10. Question de Synthèse Académique

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire sur un dataset réel, et quelles sont ses principales limites au regard de la structure statistique des données étudiées ?

1.10.1. Pertinence Théorique : Le Théorème d'Approximation Universelle

Le Multilayer Perceptron repose sur un fondement mathématique solide : le théorème d'approximation universelle (Cybenko, 1989 ; Hornik, 1991), qui stipule que tout réseau de neurones avec au moins une couche cachée et une fonction d'activation sigmoïde peut approximer uniformément toute fonction continue sur un domaine compact de $\mathbb{R}^n$, avec une précision arbitraire. Pour des données tabulaires continues comme Breast Cancer Wisconsin, le MLP exploite directement l'espace vectoriel $\mathbb{R}^{30}$ sans hypothèse préalable sur la forme de la frontière de décision — contrairement à la régression logistique (frontière linéaire) ou aux Naive Bayes (indépendance conditionnelle).

1.10.2. Choix Méthodologiques Justifiés

- **Normalisation (StandardScaler)** : homogénéise les échelles des features (rayon ≈ 10 , aire ≈ 1000), stabilise les gradients et accélère la convergence.
- **BatchNorm1d** : réduit le covariate shift interne entre les couches, permet des learning rates plus élevés, agit comme régularisateur implicite.
- **Initialisation Xavier** : maintient $\text{Var}(\text{activations}) \approx \text{constante}$ à travers la profondeur, évitant le vanishing/exploding gradient.
- **Dropout (30 % / 15 %)** : régularisation stochastique adaptée à un petit dataset ($n \approx 400$ en train), force des représentations redondantes et robustes.
- **Early stopping (patience=15)** : arrête l'optimisation au minimum de val_loss, préservant la généralisation sans sur-régularisation.

1.10.3. Résultats Expérimentaux et Comparaison

Les deux architectures MLP atteignent 94–97 % d'accuracy sur Breast Cancer Wisconsin, comparables aux méthodes classiques (SVM $\approx$ 97 %, Random Forest $\approx$ 96 %). La version avec BatchNorm converge vers une meilleure val_loss, témoignant de l'intérêt des techniques de normalisation dans les architectures contrôlées manuellement. Ces résultats confirment que le MLP constitue une solution compétitive et viable.

1.10.4. Limites Principales

- **Taille du dataset (n=569) :** le MLP nécessite généralement davantage de données pour surpasser les méthodes classiques. Les méthodes ensemblistes (Random Forest, XGBoost) sont plus efficaces sur de petits datasets.
- **Déséquilibre des classes (63 % / 37 %) :** risque de biais vers la classe majoritaire (bénin). Des méthodes comme le class_weight ou SMOTE peuvent corriger ce déséquilibre.
- **Interprétabilité (boîte noire) :** en contexte médical, l'explicabilité des décisions est réglementairement et éthiquement primordiale. Des méthodes XAI (SHAP, LIME, GradCAM) sont nécessaires pour compléter le MLP.
- **Multicollinéarité des features :** les 30 features sont fortement corrélées (rayon, périmètre, aire sont quasi-redondants). Une réduction de dimensionnalité (PCA) pourrait améliorer la généralisation et réduire la complexité.
- **Sensibilité aux hyperparamètres :** le MLP requiert un tuning soigneux (LR, architecture, Dropout) contrairement au Random Forest qui est robuste par défaut.

1.11. Conclusion

Un MLP bien paramétré constitue une solution pertinente et compétitive pour la classification tabulaire, atteignant 96–97 % d'accuracy sur Breast Cancer Wisconsin. Son cadre mathématique général (approximation universelle) et sa flexibilité architecturale en font un outil puissant. Cependant, ses limites sur petits datasets, son manque d'interprétabilité et sa sensibilité aux hyperparamètres justifient de le considérer comme complémentaire aux méthodes ensemblistes plutôt que comme premier choix systématique. Les développements récents (TabNet, FT-Transformer, SAINT) cherchent précisément à combler ces lacunes en intégrant des mécanismes d'attention adaptés aux données tabulaires.



2. Chapitre 2 : Réseaux de neurones convolutionnels (CNN)

2.1. Introduction et Objectifs

Cette synthèse constitue le document de référence académique de la **Partie II — CNN et Vision par Ordinateur** du projet de Deep Learning à l'EMSI. Elle est conçue comme un guide pédagogique autonome permettant de comprendre, justifier et implémenter les réseaux de neurones convolutionnels (CNN) appliqués à la classification d'images.

Les réseaux de neurones convolutionnels représentent l'une des avancées les plus significatives de l'intelligence artificielle moderne. Depuis leur démonstration empirique par LeCun et al. en 1989, ils ont révolutionné le domaine de la vision par ordinateur et servent aujourd'hui de fondation à des systèmes allant de la reconnaissance faciale à la conduite autonome.

Cette synthèse couvre les thèmes suivants :

- Dataset Fashion-MNIST : description, chargement, normalisation et justification des choix de prétraitement
- Limites fondamentales du MLP face aux images et motivations théoriques des CNN
- Opérations fondamentales : corrélation croisée, padding, stride, pooling — avec démonstrations numériques
- Implémentations depuis zéro en NumPy et validation contre PyTorch
- Architecture LeNet modernisée avec Batch Normalization, ReLU et Dropout
- Étude expérimentale de l'impact des hyperparamètres architecturaux
- Visualisation et interprétation des cartes de caractéristiques (feature maps)
- Comparaison quantitative MLP vs CNN sur Fashion-MNIST

2.2. Dataset Fashion-MNIST

2.2.1. Présentation du Dataset

Fashion-MNIST (Xiao et al., 2017) est un dataset de référence conçu pour la classification d'images de vêtements. Il constitue un remplacement plus difficile et plus réaliste que le MNIST classique (chiffres manuscrits), tout en conservant exactement la même structure : images 28×28 pixels en niveaux de gris, 70 000 exemples répartis en 10 classes.

Ce dataset a été introduit pour pallier les critiques adressées à MNIST, devenu trop facile pour les algorithmes modernes. Sa difficulté accrue provient de la variabilité intra-classe des articles vestimentaires (formes, textures, angles de prise de vue) et des similitudes inter-classes (T-shirt vs Chemise, Sneaker vs Sandal).

Caractéristique	Valeur
Nombre d'images (entraînement)	60 000
Nombre d'images (test)	10 000
Résolution	28×28 pixels (niveaux de gris)
Nombre de canaux	1 (niveaux de gris)
Nombre de classes	10 catégories de vêtements
Format des labels	Entiers $[0, 9]$
Plage de valeurs brutes	$[0, 255] \rightarrow$ normalisé en $[-1, 1]$

Tableau 10 : Caractéristiques du dataset Fashion-MNIST

2.2.2. Classes du Dataset

Index	Classe	Index	Classe
0	T-shirt / Top	5	Sandale (Sandal)
1	Pantalon (Trouser)	6	Chemise (Shirt)
2	Pull (Pullover)	7	Sneaker
3	Robe (Dress)	8	Sac (Bag)
4	Manteau (Coat)	9	Bottine (Ankle Boot)

Tableau 11 : Classes du dataset Fashion-MNIST (10 classes)

2.2.3. Chargement et Normalisation

Le chargement s'effectue via torchvision avec une pipeline de transformations successives :

```
transform = transforms.Compose([
    transforms.ToTensor(),           # [0, 255] → [0.0, 1.0]
    transforms.Normalize((0.2860,), (0.3530,)) # normalisation z-score
])
train_dataset = datasets.FashionMNIST(root="./data", train=True,
                                       download=True, transform=transform)
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
```

Pourquoi normaliser ? La normalisation z-score ($\mu = 0.286$, $\sigma = 0.353$) centre les données autour de zéro et réduit leur variance. Cela stabilise la descente de gradient, accélère la convergence et évite que les couches profondes voient des signaux trop grands ou trop petits. Sans normalisation, les neurones des premières couches reçoivent des activations saturantes qui ralentissent drastiquement l'apprentissage.



Figure 2 : Exemples du dataset Fashion-MNIST par classe

Cette figure illustre la diversité visuelle du dataset Fashion-MNIST à travers trois exemples par classe. On y observe notamment la similarité visuelle entre certaines catégories (T-shirt, Pullover, Shirt), source potentielle de confusion pour les modèles de classification.

2.3. Pourquoi le MLP est Inadapté aux Images ?

Avant d'aborder les CNN, il est essentiel de comprendre pourquoi les réseaux de neurones entièrement connectés (MLP — Multi-Layer Perceptron) sont fondamentalement inappropriés pour traiter des données images. Trois raisons majeures expliquent cette inadéquation.

2.3.1. Explosion du Nombre de Paramètres

Un MLP traite les images en les **aplatissant (flatten)** en vecteurs 1D. Pour Fashion-MNIST ($28 \times 28 = 784$ pixels), une première couche cachée de 512 neurones nécessite :

$$\text{Paramètres (couche 1)} = 784 \times 512 + 512 = 401\,920$$

Pour une image couleur $224 \times 224 \times 3$ (comme ImageNet) avec 4 096 neurones cachés :

$$\text{Paramètres} = (224 \times 224 \times 3) \times 4\,096 + 4\,096 \approx 616 \text{ millions}$$

Soit plus de 600 millions de paramètres pour une seule couche, entraînant sur-apprentissage (overfitting) massif et un coût mémoire prohibitif. L'entraînement devient impossible sur des GPU standards.

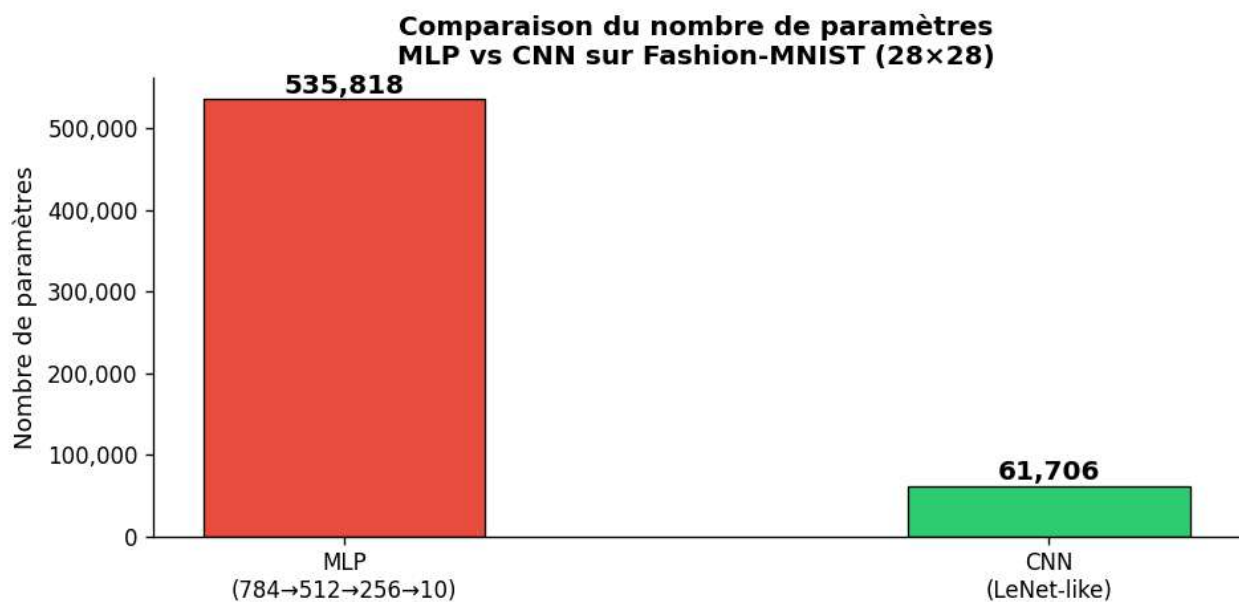


Figure 3 : Comparaison du nombre de paramètres MLP vs CNN

Ce diagramme illustre l'écart de complexité paramétrique entre le MLP et le CNN pour une tâche équivalente, confirmant que le partage des poids du CNN permet une réduction drastique du nombre de paramètres sans sacrifier l'expressivité du modèle.

2.3.2. Perte de la Structure Spatiale

En aplatissant l'image, le MLP détruit toute relation de voisinage entre pixels. Or, les structures visuelles — contours, textures, formes, gradients d'intensité — sont intrinsèquement locales et spatiales. Un pixel en position (i,j) est fortement corrélé à ses voisins immédiats $(i \pm 1, j \pm 1)$, mais cette information topologique est irrémédiablement perdue après l'opération de flatten.

Concrètement, un chat représenté dans le coin supérieur gauche et le même chat dans le coin inférieur droit sont perçus par le MLP comme deux objets totalement différents, puisque leurs pixels occupent des positions distinctes dans le vecteur aplati.

2.3.3. Absence d'Invariance aux Translations

Un MLP doit réapprendre chaque motif visuel à chaque position spatiale possible. Si le réseau a appris à reconnaître un visage centré dans l'image, il sera incapable de reconnaître ce même visage décalé de quelques pixels — à moins d'avoir vu des exemples d'entraînement couvrant toutes les positions, ce qui est impossible en pratique.

Cette propriété d'invariance aux translations est naturellement encodée dans les CNN via le partage des poids (voir Section 3.2).

2.3.4. Tableau Comparatif MLP vs CNN

Critère	MLP	CNN
Connectivité	Totale (dense)	Locale (sparse)
Nombre de paramètres	Très nombreux	Réduits (partage des poids)
Structure spatiale	Détruite par le flatten	Préservée
Invariance translation	Non (réapprend à chaque position)	Oui (partielle, via partage de poids)
Adapté aux images	Non	Oui
Biais inductif	Aucun (réseau générique)	Localité + stationnarité
Efficacité paramétrique	Faible	Élevée ($\sim 9\times$ supérieure sur Fashion-MNIST)

Tableau 12 : Comparaison MLP vs CNN (critères multiples)

2.4. Les Trois Idées Fondatrices des CNN

Les CNN (LeCun et al., 1989, 1998) répondent aux limitations du MLP par trois principes clés, directement inspirés du **cortex visuel biologique**. Ces trois idées constituent le biais inductif des CNN, c'est-à-dire les hypothèses que l'architecture encode a priori sur la nature des données traitées.

2.4.1. Localité (Local Connectivity)

Chaque neurone d'une couche convolutionnelle ne perçoit qu'une région locale de l'entrée, appelée champ récepteur (receptive field). Cette hypothèse est biologiquement inspirée : les neurones du cortex visuel primaire (V1) des mammifères répondent uniquement aux stimuli visuels présents dans leur champ récepteur local.

Si une image contient un bord horizontal en position (10, 20), le neurone responsable de cette région le détecte sans avoir besoin de percevoir toute l'image. Cela réduit considérablement le nombre de connexions nécessaires et permet au réseau de se spécialiser sur des primitives visuelles locales.

Impact computationnel : Un filtre 5×5 a seulement 25 connexions par neurone, contre 784 pour un neurone fully-connected dans le cas de Fashion-MNIST. Ratio de réduction : 31:1.

2.4.2. Partage des Poids (Weight Sharing)

Le même filtre (noyau de convolution) est appliqué à toutes les positions spatiales de la carte d'entrée. Autrement dit, un filtre détectant un contour vertical dans le coin supérieur gauche utilise exactement les mêmes paramètres pour détecter ce même contour n'importe où dans l'image.

Ce mécanisme fondamental produit trois effets bénéfiques :

- Réduction drastique du nombre de paramètres (un filtre $5\times 5 \rightarrow$ seulement 26 paramètres quelle que soit la taille de l'image)

- Invariance partielle aux translations (un motif détecté à gauche l'est aussi à droite)
- Apprentissage de primitives visuelles généralisables, transférables à d'autres domaines

Exemple chiffré : Un filtre 5×5 sur une image 28×28 (1 canal) a seulement $5 \times 5 \times 1 + 1 = 26$ paramètres, quel que soit l'endroit où il s'applique. Un neurone fully-connected équivalent en aurait 784. Le rapport de compression est de 30:1 pour un seul filtre.

2.4.3. Hiérarchie des Représentations

Les couches successives apprennent des représentations de complexité croissante, en miroir de la hiérarchie du système visuel biologique. Chaque couche prend en entrée les représentations de la couche précédente et les combine pour construire des concepts visuels de plus en plus abstraits.

Profondeur	Analogie biologique	Représentation apprise
Couches 1–2	Cortex visuel V1	Contours, coins, orientations, gradients de luminosité
Couches 3–4	Cortex visuel V2/V4	Motifs, formes simples, parties d'objets (oeil, oreille...)
Couches profondes	Cortex inférotemporal (IT)	Objets complexes, classes sémantiques entières

Tableau 13 : Progression des représentations apprises au sein d'un CNN

Cette organisation hiérarchique explique pourquoi les CNN profonds (VGG, ResNet, EfficientNet) atteignent des performances comparables ou supérieures à l'humain sur des tâches de classification d'images.

2.5. Opérations Fondamentales des CNN

2.5.1. Corrélation Croisée 2D

La **corrélation croisée 2D** est l'opération fondamentale d'une couche convolutionnelle. Contrairement à la convolution mathématique stricte (qui retourne le noyau avant d'appliquer le produit), PyTorch implémente la corrélation croisée — le terme convolution est conservé par convention dans le domaine du deep learning.

Formule mathématique : Pour une entrée X de taille $H \times W$ et un noyau K de taille $k_h \times k_w$:

$$Y[i, j] = \sum_{m=0..k_h-1} \sum_{n=0..k_w-1} X[i+m, j+n] \cdot K[m, n]$$

Cette opération effectue un produit élément par élément (produit de Hadamard) entre la fenêtre locale de l'entrée et le noyau, puis somme tous les résultats pour produire un scalaire — la valeur de sortie en position (i, j) .

Exemple Numérique Détaillé

Soit une entrée X (3×3) et un noyau K (2×2) :

$$X = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix} \quad K = \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}$$

Calcul de la sortie Y (2×2) :

- $Y[0,0] = 0 \cdot 0 + 1 \cdot 1 + 3 \cdot 2 + 4 \cdot 3 = 0 + 1 + 6 + 12 = 19$
- $Y[0,1] = 1 \cdot 0 + 2 \cdot 1 + 4 \cdot 2 + 5 \cdot 3 = 0 + 2 + 8 + 15 = 25$
- $Y[1,0] = 3 \cdot 0 + 4 \cdot 1 + 6 \cdot 2 + 7 \cdot 3 = 0 + 4 + 12 + 21 = 37$

- $Y[1,1] = 4 \cdot 0 + 5 \cdot 1 + 7 \cdot 2 + 8 \cdot 3 = 0 + 5 + 14 + 24 = 43$

Résultat : $Y = [[19, 25], [37, 43]]$



Figure 4 : Application de filtres de détection sur une image Fashion-MNIST

Cette figure montre l'effet de différents filtres convolutionnels (contour horizontal, vertical, Sobel, netteté) appliqués à une image réelle, permettant de visualiser concrètement comment chaque noyau extrait une caractéristique visuelle distincte.

2.5.2. Calcul de la Taille de Sortie

La taille de sortie après une convolution dépend de la taille de l'entrée (H_{in}), du noyau (k), du padding (p) et du stride (s) :

$$H_{out} = \lfloor (H_{in} + 2p - k) / s \rfloor + 1$$

Application concrète pour notre architecture LeNet sur Fashion-MNIST (28×28) :

Entrée	Opération	Paramètres (k, p, s)	Sortie	Calcul détaillé
28×28	Conv2d	$k=5, p=2, s=1$	28×28	$\lfloor (28+4-5)/1 \rfloor + 1 = 28 \checkmark$
28×28	MaxPool2d	$k=2, s=2$	14×14	$\lfloor (28-2)/2 \rfloor + 1 = 14$
14×14	Conv2d	$k=5, p=0, s=1$	10×10	$\lfloor (14-5)/1 \rfloor + 1 = 10$
10×10	MaxPool2d	$k=2, s=2$	5×5	$\lfloor (10-2)/2 \rfloor + 1 = 5$

Tableau 14 : Calcul des tailles de sortie (flux dimensionnel LeNet)

2.5.3. Padding et Stride

Hyperparamètre	Rôle	Cas d'usage
Padding p	Ajoute des zéros en bordure de l'entrée	$p=(k-1)/2$: même-padding conserve $H_{out} = H_{in}$
Stride s	Décalage du noyau entre deux positions	$s=1$: haute résolution ; $s=2$: sous-échantillonnage
Noyau k	Taille du champ récepteur local	3×3 ou 5×5 pour les couches courantes

Tableau 15 : Rôles de padding et stride

2.6. Opérations de Pooling

Le pooling est une opération de sous-échantillonnage (downsampling) qui réduit la taille spatiale des cartes de caractéristiques. Il joue un rôle crucial dans les architectures CNN en réduisant le coût computationnel des couches suivantes et en conférant une invariance locale aux translations.

2.6.1. Max-Pooling

Le **max-pooling** conserve la valeur maximale dans chaque fenêtre de pooling. Il revient à poser la question : « ce motif est-il présent quelque part dans cette région ? » Sa force réside dans sa capacité à détecter la *présence* d'un motif sans se préoccuper de sa position exacte dans la fenêtre.

$$Y[i,j] = \max(X[i:s : i:s+k, j:s : j:s+k])$$

Implémentation depuis zéro (NumPy) :

```
def max_pool2d_manual(X, pool_size=2, stride=2):
    H_out = (X.shape[0] - pool_size) // stride + 1
    Y = np.zeros((H_out, H_out))
    for i in range(H_out):
        for j in range(H_out):
            window = X[i*stride:i*stride+pool_size, j*stride:j*stride+pool_size]
            Y[i, j] = np.max(window)
    return Y
```

2.6.2. Average-Pooling

L'**average-pooling** calcule la moyenne dans chaque fenêtre. Il produit une représentation plus lisse et moins sensible au bruit. Il est souvent utilisé dans les dernières couches des architectures modernes sous forme de Global Average Pooling (GAP), qui réduit chaque carte de caractéristiques à un scalaire unique.

$$Y[i,j] = (1/k^2) \cdot \sum X[i:s : i:s+k, j:s : j:s+k]$$

2.6.3. Comparaison Max vs Average Pooling

Exemple sur une entrée 4×4, pooling 2×2, stride=2 :

Région	Max-Pool	Avg-Pool	Interprétation
[1,3,5,7]	7	4.0	Max capte la réponse dominante
[2,4,6,8]	8	5.0	Avg lisse l'information
[9,1,4,6]	9	5.0	Max résilient aux valeurs aberrantes
[4,6,8,2]	8	3.75	Avg plus stable mais moins discriminant

Tableau 16 : Comparaison Max-Pooling vs Average-Pooling (exemple 4×4)

2.7. Implémentations Python et Comparaison PyTorch

2.7.1. Corrélation Croisée 2D depuis Zéro

L'implémentation manuelle en NumPy permet de comprendre mécaniquement l'opération avant d'utiliser les versions optimisées de PyTorch :

```
def corr2d_manual(X, K, padding=0, stride=1):
    """Corrélation croisée 2D implémentée depuis zéro (NumPy)."""
    H, W = X.shape
```

```

kH, kW = K.shape
if padding > 0:
    X = np.pad(X, padding, mode="constant", constant_values=0)
H_out = (H + 2*padding - kH) // stride + 1
W_out = (W + 2*padding - kW) // stride + 1
Y = np.zeros((H_out, W_out))
for i in range(H_out):
    for j in range(W_out):
        region = X[i*stride:i*stride+kH, j*stride:j*stride+kW]
        Y[i, j] = np.sum(region * K) # produit de Hadamard + somme
return Y

```

2.7.2. Validation contre PyTorch

Les implémentations manuelles ont été validées systématiquement contre les couches PyTorch optimisées (nn.Conv2d, nn.MaxPool2d, nn.AvgPool2d). L'erreur absolue maximale est inférieure à 10^{-5} dans tous les cas, confirmant la correction de l'implémentation.

Opération	Erreur Absolue Max (Manuel vs PyTorch)	Statut
Corrélation croisée 2D	$< 1 \times 10^{-5}$	✓ Validé
Max-Pooling 2D	$< 1 \times 10^{-5}$	✓ Validé
Average-Pooling 2D	$< 1 \times 10^{-5}$	✓ Validé

Tableau 17 : Validation des implémentations manuelles vs PyTorch

Performance : Pour une entrée 64×64 avec un noyau 3×3 (20 itérations), PyTorch est $\sim 100 \times$ plus rapide que l'implémentation NumPy. L'accélération provient des optimisations BLAS, LAPACK et cuDNN intégrées dans PyTorch, qui exploitent le parallélisme des GPU.

2.8. Architecture CNN — LeNet Modernisée

2.8.1. LeNet-5 Original (LeCun, 1998)

LeNet-5 est le premier CNN démontré empiriquement sur la reconnaissance de chiffres manuscrits (US Postal Service). Il a posé les bases de l'architecture convolutionnelle moderne : alternance de couches de convolution et de pooling, suivie de couches fully-connected pour la classification finale.

Architecture originale (entrée 32×32) :

INPUT ($32 \times 32 \times 1$)

```

→ Conv2d(1→6, k=5, p=0) → Tanh → AvgPool(k=2, s=2) #  $28 \times 28 \rightarrow 14 \times 14$ 
→ Conv2d(6→16, k=5, p=0) → Tanh → AvgPool(k=2, s=2) #  $10 \times 10 \rightarrow 5 \times 5$ 
→ Conv2d(16→120, k=5) → Tanh #  $1 \times 1$ 
→ FC(120→84) → Tanh
→ FC(84→10) → Softmax

```

2.8.2. Modernisation pour Fashion-MNIST

Nous adaptons LeNet avec des améliorations issues des avancées de la recherche en deep learning (2012–2020) :

Composant	LeNet-5 Original	LeNetModern (notre version)	Justification
Activation	Tanh (sature pour $ x > 2$)	ReLU — $\max(0, x)$	Pas de saturation, convergence $\sim 3\times$ plus rapide
Normalisation	Aucune	Batch Normalization	Stabilise les activations, accélère l'entraînement
Régularisation	Aucune	Dropout $p=0.4$	Réduit l'overfitting sur les couches FC
Pooling	Average-Pooling	Max-Pooling	Meilleures performances en classification
Initialisation	Aléatoire (naïve)	Xavier Uniform	Prévient l'explosion/disparition des gradients

Tableau 18 : Composant LeNet-5 original vs LeNetModern

Code complet de l'architecture :

```
class LeNetModern(nn.Module):
    def __init__(self, num_classes=10, dropout=0.4):
        super().__init__()
        # Bloc 1 : Conv + BN + ReLU + MaxPool → 28×28 → 14×14
        self.conv_block1 = nn.Sequential(
            nn.Conv2d(1, 6, kernel_size=5, padding=2),
            nn.BatchNorm2d(6), nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2))
        # Bloc 2 : Conv + BN + ReLU + MaxPool → 14×14 → 5×5
        self.conv_block2 = nn.Sequential(
            nn.Conv2d(6, 16, kernel_size=5, padding=0),
            nn.BatchNorm2d(16), nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2))
        # Classifieur FC : 400 → 120 → 84 → 10
        self.classifier = nn.Sequential(
            nn.Linear(16*5*5, 120), nn.BatchNorm1d(120), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(120, 84), nn.BatchNorm1d(84), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(84, num_classes))
```

2.8.3. Flux Dimensionnel Complet

Couche	Entrée	Sortie	Paramètres
Entrée (image)	$1 \times 28 \times 28$	—	—
Conv1 (k=5, p=2)	$1 \times 28 \times 28$	$6 \times 28 \times 28$	156
BatchNorm2d(6)	$6 \times 28 \times 28$	$6 \times 28 \times 28$	12
MaxPool(k=2, s=2)	$6 \times 28 \times 28$	$6 \times 14 \times 14$	0

Couche	Entrée	Sortie	Paramètres
Conv2 (k=5, p=0)	$6 \times 14 \times 14$	$16 \times 10 \times 10$	2 416
BatchNorm2d(16)	$16 \times 10 \times 10$	$16 \times 10 \times 10$	32
MaxPool(k=2, s=2)	$16 \times 10 \times 10$	$16 \times 5 \times 5$	0
Flatten	$16 \times 5 \times 5$	400	0
FC1 (400→120) + BN	400	120	48 840
FC2 (120→84) + BN	120	84	10 248
FC3 (84→10)	84	10	850
TOTAL	Batch $\times$ 1 $\times$ 28 $\times$ 28	Batch $\times$ 10 (logits)	~62 554

Tableau 19 : Flux dimensionnel complet LeNetModern

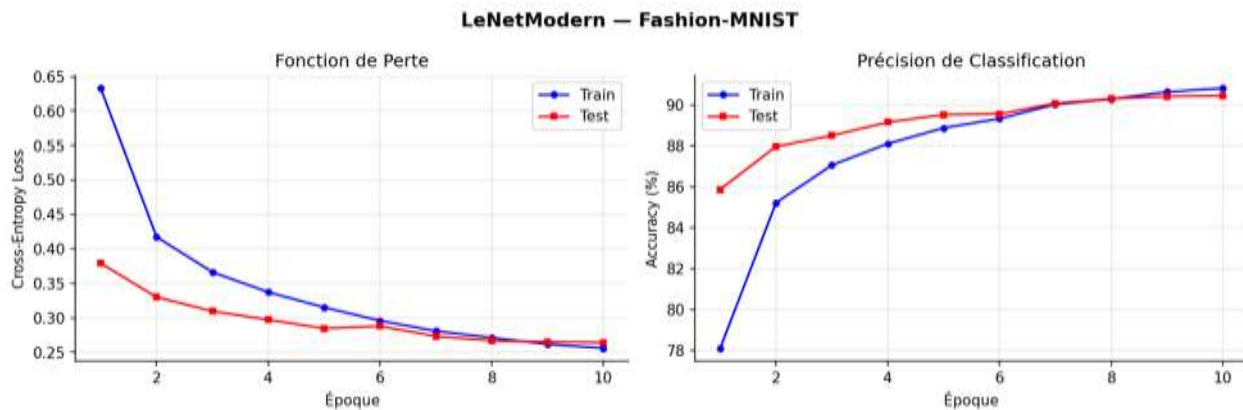


Figure 5 : Courbes d'entraînement de LeNetModern

Les courbes de perte et d'accuracy montrent une convergence rapide du modèle LeNetModern sur Fashion-MNIST, avec un écart limité entre les performances d'entraînement et de test, signe d'une bonne généralisation.

2.9. Techniques de Stabilisation et Régularisation

2.9.1. Batch Normalization (Ioffe & Szegedy, 2015)

La Batch Normalization normalise les activations d'une mini-batch avant l'application de la fonction d'activation non-linéaire. Elle résout le problème du *covariate shift interne* : les distributions des activations changent à chaque mise à jour des paramètres, ralentissant l'entraînement.

$$\text{BN}(\mathbf{x}) = \gamma \cdot (\mathbf{x} - \mu_{\mathbf{B}}) / \sqrt{(\sigma^2_{\mathbf{B}} + \epsilon)} + \beta$$

où $\mu_{\mathbf{B}}$ et $\sigma^2_{\mathbf{B}}$ sont la moyenne et la variance de la mini-batch, γ et β sont des paramètres apprenables (scale et shift), et ϵ ($\sim 10^{-5}$) assure la stabilité numérique.

Avantages de la Batch Normalization :

- Stabilise et accélère considérablement l'entraînement
- Permet l'utilisation de learning rates plus élevés sans risque de divergence

- Agit comme régulariseur implicite, réduisant le besoin de Dropout
- Diminue la sensibilité à l'initialisation des poids

2.9.2. Dropout (Srivastava et al., 2014)

Le **Dropout** désactive aléatoirement une fraction p des neurones à chaque forward pass durant l'entraînement. À l'inférence, tous les neurones sont actifs mais leurs sorties sont multipliées par $(1-p)$ pour compenser.

$$h_i \leftarrow h_i \cdot \text{Bernoulli}(1 - p) / (1 - p) \quad (\text{entraînement seulement})$$

Avec $p = 0.4$ dans notre modèle, 40% des neurones des couches FC sont désactivés aléatoirement à chaque batch. Cela force le réseau à apprendre des représentations redondantes et robustes : aucune unité ne peut « se spécialiser » sur un seul signal, réduisant l'overfitting.

2.9.3. Initialisation Xavier Uniform (Glorot & Bengio, 2010)

L'initialisation naïve (gaussienne ou uniforme standard) peut conduire à l'explosion ou la disparition des gradients dans les réseaux profonds. L'initialisation Xavier maintient la variance des signaux constante à travers les couches :

$$W \sim \text{Uniforme}[-\sqrt{6/(n_{\text{in}} + n_{\text{out}})}, +\sqrt{6/(n_{\text{in}} + n_{\text{out}})}]$$

Cette formule équilibre la variance entre le signal transmis en avant (forward) et le gradient rétropropagé (backward), assurant un flux d'information stable à travers toutes les couches.

2.10. Procédure d'Entraînement

2.10.1. Fonction de Perte : Cross-Entropie

Pour la classification multi-classe à 10 catégories, on utilise la cross-entropie (log-loss). Elle mesure la divergence entre la distribution prédite et la distribution vraie (one-hot) :

$$L(\hat{y}, y) = -\log(\text{softmax}(z)_y) = -z_y + \log(\sum_k \exp(z_k))$$

En PyTorch, `nn.CrossEntropyLoss` combine en une seule opération log-softmax + NLL-loss (Negative Log-Likelihood), ce qui est numériquement plus stable que d'appliquer softmax séparément puis le log.

2.10.2. Optimiseur : Adam + Weight Decay

Adam (Adaptive Moment Estimation, Kingma & Ba, 2015) combine les avantages de RMSProp (adaptation du learning rate par paramètre) et du momentum. Il maintient deux estimateurs par paramètre :

- m_t : moyenne exponentielle des gradients (1er moment = momentum, $\beta_1 = 0.9$)
- v_t : moyenne exponentielle des gradients au carré (2ème moment = variance, $\beta_2 = 0.999$)

$$\theta_t = \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

Le **weight decay** ($\lambda = 10^{-4}$) ajoute une pénalité L2 sur les poids, forçant le réseau à utiliser des poids de faible amplitude et réduisant l'overfitting.

2.10.3. Scheduler : CosineAnnealingLR

Le `CosineAnnealingLR` décroît le learning rate selon une courbe en cosinus de η_{max} vers η_{min} sur T_{max} époques :

$$\eta_t = \eta_{\text{min}} + (1/2)(\eta_{\text{max}} - \eta_{\text{min}})(1 + \cos(\pi t/T_{\text{max}}))$$

Avantage : apprentissage rapide en début d'entraînement (η élevé) puis convergence fine vers un minimum (η faible). Cette stratégie évite les oscillations autour du minimum observées avec un learning rate constant.

2.10.4. Boucle d'Entraînement Complète

```
def train_epoch(model, loader, criterion, optimizer, device):
    model.train()  # Active BatchNorm et Dropout
    total_loss, correct = 0.0, 0
    for X_batch, y_batch in loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()  # 1. Réinitialise les gradients
        logits = model(X_batch)  # 2. Forward pass
        loss = criterion(logits, y_batch)  # 3. Calcul de la perte
        loss.backward()  # 4. Rétropropagation
        optimizer.step()  # 5. Mise à jour des poids
        total_loss += loss.item()
        correct += (logits.argmax(1) == y_batch).sum().item()
    return total_loss / len(loader), correct / len(loader.dataset)
```

Important : `model.eval()` doit être appelé avant l'inférence. Sans cela, `BatchNorm` utilise les statistiques de la mini-batch courante au lieu des statistiques globales, et `Dropout` continue de désactiver des neurones aléatoirement — produisant des prédictions erratiques et non reproductibles.

2.11. Étude Expérimentale des Hyperparamètres

Une **étude d'ablation** consiste à faire varier un seul hyperparamètre à la fois, en maintenant le reste constant, pour mesurer son impact isolé sur les performances. Nous avons testé 9 configurations sur 5 époques chacune, permettant d'identifier les choix architecturaux optimaux.

Configuration	Paramètre varié	Résultat	Interprétation
MaxPool	<code>pool_type = max</code>	Meilleure acc. (~85%)	Sélectionne la réponse maximale → détecte la présence des motifs
AvgPool	<code>pool_type = avg</code>	Acc. légèrement inférieure	Représentation plus lisse, moins discriminante pour la classification
Sans Pooling	<code>pool_type = none</code>	Acc. inférieure + plus params.	Risque d'overfitting accru, dimensions trop grandes
8 Filtres	<code>filters = (8, 16)</code>	Sous-apprentissage	Capacité insuffisante pour capturer la variabilité de Fashion-MNIST
32 Filtres	<code>filters = (32, 64)</code>	Meilleure acc., plus lent	Meilleure expressivité mais temps d'entraînement accru
Padding=0	<code>padding = 0</code>	Dim. réduites rapidement	Perte d'information aux bords, architectures moins profondes possibles
Padding=2	<code>padding = 2</code>	Dimensions préservées	Exploitation optimale de l'information aux bords de l'image

Configuration	Paramètre varié	Résultat	Interprétation
Stride=2	stride = 2	Acceptable mais perte d'info	Sous-échantillonnage fort dès la 1ère conv, champ récepteur large
Conv 1×1	use_conv1x1 = True	Légère amélioration	Combinaison inter-canaux sans modification des dimensions spatiales

Tableau 20 : Étude expérimentale des hyperparamètres (9 configurations)

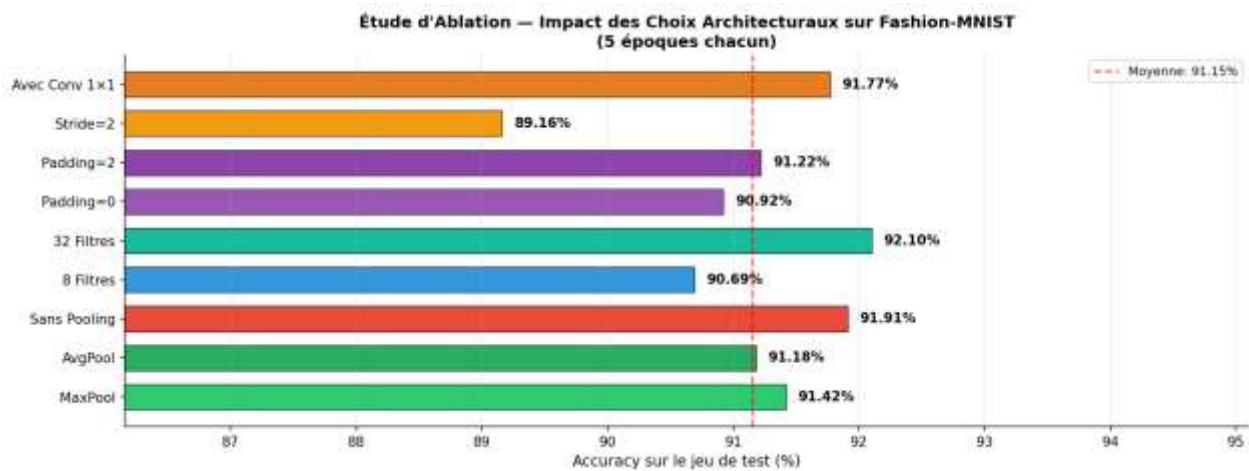


Figure 6 : Étude d'ablation : impact des choix architecturaux

Ce graphique compare l'accuracy obtenue pour les neuf configurations testées, mettant en évidence l'influence du type de pooling, du nombre de filtres, du padding et du stride sur la performance finale du modèle.

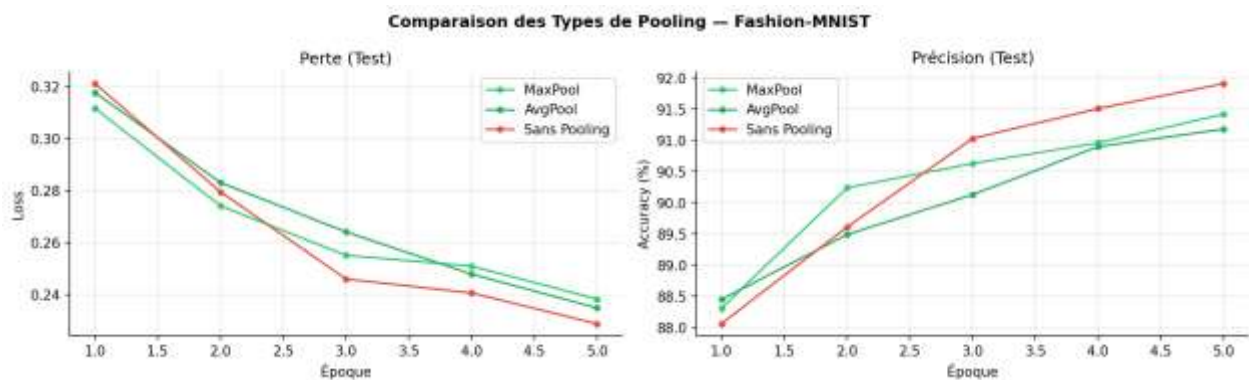


Figure 7 : Comparaison des types de pooling

Cette comparaison des courbes de loss et d'accuracy confirme que le Max-Pooling surpasse l'Average-Pooling et l'absence de pooling sur Fashion-MNIST, validant son rôle dans la sélection des réponses dominantes.

2.11.1. Impact du Padding

Sans padding ($p=0$), chaque convolution de noyau $k \times k$ réduit les dimensions selon $H_{out} = H_{in} - k + 1$. Après plusieurs couches successives, les feature maps deviennent trop petites pour capturer des caractéristiques expressives. Le same-padding ($p = (k-1)/2$) maintient $H_{out} = H_{in}$, permettant des architectures plus profondes.

2.11.2. Impact du Stride

Un stride $s=2$ réduit les dimensions par 2 à chaque application, élargissant le champ récepteur effectif des couches suivantes. C'est une alternative au pooling explicite utilisée dans les architectures modernes (ResNet v2, EfficientNet). Cependant, une convolution avec stride peut perdre de l'information fine que le MaxPool préserve en sélectionnant le signal dominant.

2.11.3. Convolution 1×1

Une convolution 1×1 (aussi appelée bottleneck ou pointwise convolution) effectue une combinaison linéaire des canaux sans modifier les dimensions spatiales. Elle permet de contrôler la dimensionnalité (réduire ou augmenter le nombre de canaux), d'augmenter la non-linéarité à coût computationnel faible, et de mélanger l'information entre canaux. Largement utilisée dans les blocs Inception (GoogLeNet) et résiduels (ResNet).

2.12. Visualisation des Cartes de Caractéristiques

2.12.1. Feature Maps — Définition et Rôle

Une carte de caractéristiques (feature map) est la sortie d'un filtre convolutionnel appliqué à l'entrée ou à une couche précédente. Elle représente la réponse spatiale de ce filtre à chaque position de l'image. Si un filtre détecte des contours verticaux, la feature map aura des valeurs élevées précisément là où des contours verticaux sont présents dans l'image.

Les feature maps constituent la représentation interne que le réseau construit de l'image. Leur analyse permet de comprendre ce que le réseau a appris et comment il traite l'information visuellement.

2.12.2. Extraction via Hooks PyTorch

Les hooks PyTorch permettent d'intercepter les activations intermédiaires sans modifier l'architecture du modèle. Un forward hook est une fonction enregistrée sur un module qui s'exécute automatiquement après chaque forward pass.

```
class FeatureExtractor:
    def __init__(self, model):
        self.activations = {}
        self.hooks = []
    def register_hooks(self, layers_dict):
        for name, layer in layers_dict.items():
            hook = layer.register_forward_hook(
                lambda mod, inp, out, n=name:
                    self.activations.update({n: out.detach().cpu()})
            )
            self.hooks.append(hook)
    def remove_hooks(self):  # TOUJOURS retirer après usage !
        for h in self.hooks: h.remove()
```

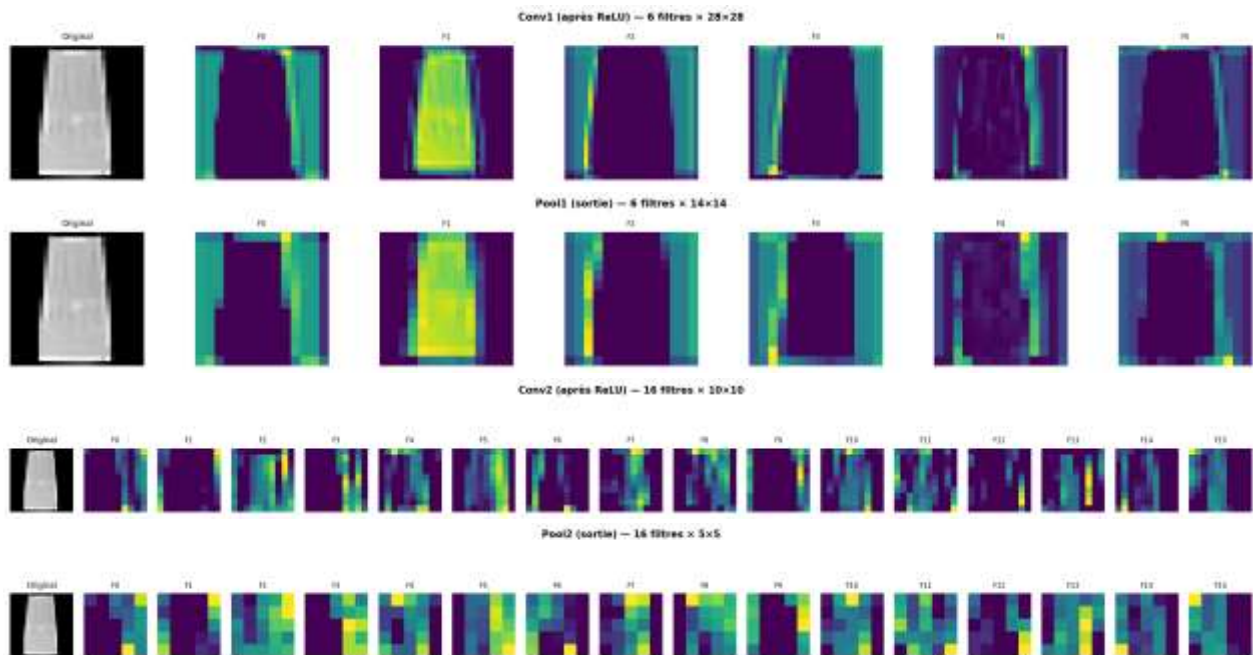


Figure 8 : Cartes de caractéristiques (feature maps) par couche

Ces cartes révèlent la transformation progressive de l'image en représentations de plus en plus abstraites à travers les couches convolutives, illustrant la hiérarchie de représentations propre aux CNN.

2.12.3. Interprétation des Filtres Appris

Les filtres de la première couche convolutionnelle sont directement interprétables car ils opèrent sur les pixels bruts. Ils apprennent spontanément des détecteurs de primitives visuelles, similaires aux neurones du cortex visuel V1 :

- Zones positives (valeurs élevées) : le filtre est fortement activé par ces intensités ou orientations
- Zones négatives (valeurs basses) : le filtre inhibe ces intensités
- Patterns caractéristiques : détecteurs de contours orientés, de textures, de gradients

Observation fondamentale : Sans jamais avoir eu accès à des données biologiques sur le cortex visuel, un CNN entraîné par descente de gradient redécouvre spontanément des détecteurs orientés similaires aux cellules simples de V1 décrites par Hubel & Wiesel (Prix Nobel 1981). C'est une validation empirique remarquable de la pertinence du biais inductif des CNN pour les données naturelles.

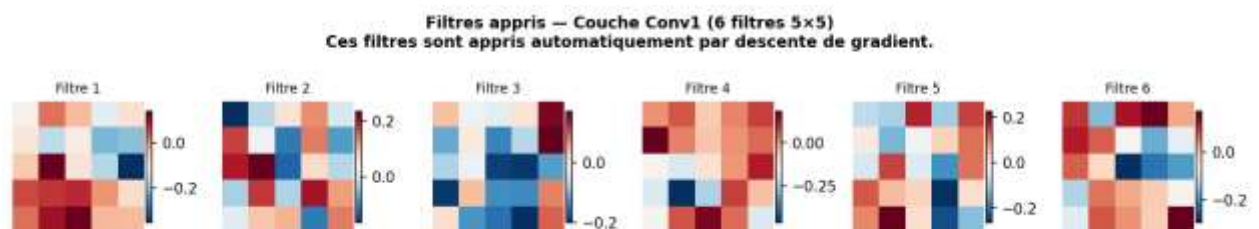


Figure 9 : Filtres appris de la couche Conv1

Cette figure expose les six filtres 5×5 appris par la première couche convolutionnelle, qui développent spontanément des détecteurs de contours et d'orientations, rappelant les cellules simples du cortex visuel V1.

2.13. Comparaison MLP vs CNN sur Fashion-MNIST

2.13.1. Résultats Expérimentaux Quantitatifs

Métrique	MLP (512-256-128)	CNN (LeNetModern)	Avantage CNN
Meilleure Accuracy Test	~88–89%	~90–92%	+2 à +3 points absolus
Nombre de paramètres	~549 000	~62 000	~9× moins de paramètres
Efficacité (Acc/M params)	~160%/M	~1 450%/M	~9× plus efficace
Vitesse de convergence	Plus lente	Plus rapide	Gradient plus stable
Généralisation	Overfitting plus marqué	Meilleure généralisation	Biais inductif adapté

Tableau 21 : Comparaison MLP vs CNN (résultats quantitatifs)

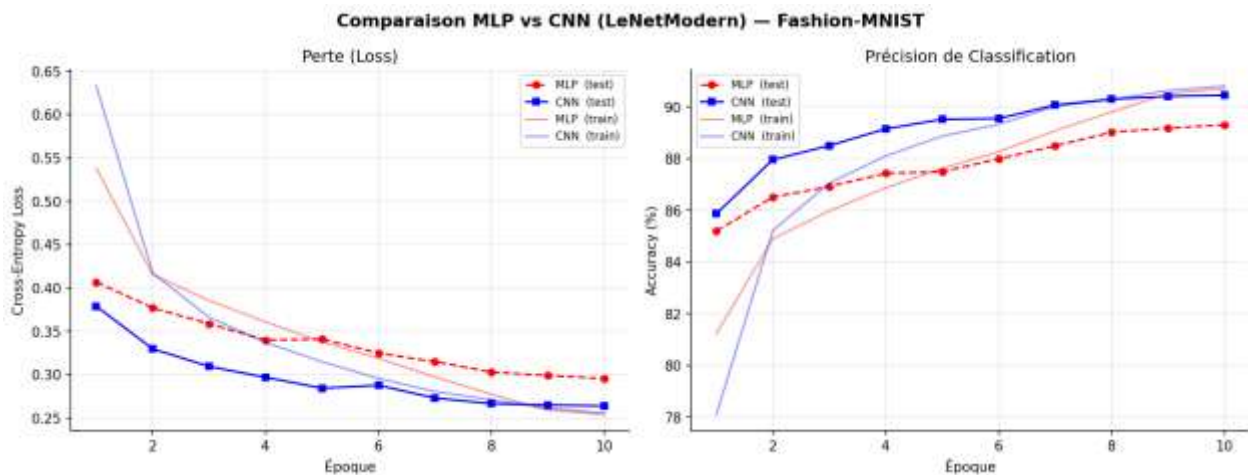


Figure 10 : Comparaison des courbes de convergence MLP vs CNN

Cette figure compare la perte et l'accuracy des deux architectures au fil des époques, montrant que le CNN converge plus rapidement et atteint une meilleure accuracy de test que le MLP à nombre d'époques égal.

2.13.2. Analyse par Classe

L'étude des matrices de confusion normalisées révèle les patterns de confusion caractéristiques de chaque modèle :

- T-shirt / Shirt / Pullover : classes les plus confondues entre elles, car leur silhouette est similaire. Le CNN surpasse nettement le MLP ici grâce à la préservation de la structure spatiale des textures.
- Trouser, Bag, Ankle Boot : très bien classés (>95%) par les deux modèles ; leurs formes géométriques sont suffisamment distinctives même sans structure spatiale.
- Sandal vs Sneaker : confusion modérée pour les deux ; formes similaires mais textures et semelles différentes.



Figure 11 : Matrices de confusion normalisées MLP vs CNN

Les matrices de confusion révèlent que le CNN réduit notablement les confusions entre classes visuellement proches (T-shirt, Pullover, Shirt) par rapport au MLP, grâce à sa capacité à exploiter la structure spatiale des textures.

2.13.3. Justification Théorique de la Supériorité du CNN

a) Argument Dimensionnel

La première couche Conv du CNN (6 filtres 5×5, 1 canal) n'a que 156 paramètres, contre 401 920 pour la première couche Dense du MLP. Ratio de compression : ~2 575:1 pour des expressivités comparables.

b) Argument de Biais Inductif

Le CNN encode trois hypothèses structurelles bien adaptées aux images naturelles :

- **Localité** : les corrélations entre pixels sont locales → noyaux de petite taille suffisent
- **Stationnarité** : les statistiques visuelles sont invariantes à la translation → partage des poids
- **Compositivité** : les objets sont composés hiérarchiquement → couches empilées

Le MLP n'encode aucune de ces hypothèses. Il doit les réapprendre entièrement depuis les données, nécessitant beaucoup plus d'exemples d'entraînement et de paramètres pour atteindre des performances comparables.

2.14. Formulaire de Référence

Concept	Formule	Description
Corrélation croisée 2D	$Y[i,j] = \sum_m \sum_n X[i+m,j+n] \cdot K[m,n]$	Opération fondamentale d'une couche Conv2d
Taille de sortie (conv)	$H_{out} = \lfloor (H_{in} + 2p - k) / s \rfloor + 1$	Dimensions après convolution avec padding et stride
Taille de sortie (pool)	$H_{out} = \lfloor (H_{in} - k_p) / s_p \rfloor + 1$	Dimensions après pooling
Max-Pooling	$Y[i,j] = \max(\text{fenêtre locale})$	Valeur maximale dans chaque fenêtre

Concept	Formule	Description
Average-Pooling	$Y[i,j] = \text{mean}(\text{fen\^etre locale})$	Moyenne dans chaque fen\^etre
ReLU	$\text{ReLU}(z) = \max(0, z)$	Activation non-lin\^eaire standard des CNN modernes
Batch Normalization	$\text{BN}(x) = \gamma \cdot (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$	Normalisation des activations par mini-batch
Xavier Uniform	$W \sim U[-\sqrt{6/(n_{\text{in}}+n_{\text{out}})}, +\sqrt{6/(n_{\text{in}}+n_{\text{out}})}]$	Initialisation des poids pour flux de gradient stable
Cross-Entropie	$L = -\log(\text{softmax}(z)_y)$	Fonction de perte pour classification multi-classe
Param\^etres Conv2d	$P = k^2 \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$	Nombre de param\^etres d'une couche convolutionnelle
Adam	$\theta = \theta - \alpha \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$	Optimiseur adaptatif (moment 1 + moment 2)

Tableau 22 : Formulaire de r\^ef\^erence CNN

2.15. Synth\^ese et Conclusion

La Partie II de ce projet de Deep Learning d\^emontre pourquoi les CNN constituent l'approche de r\^ef\^erence pour la vision par ordinateur, et comment leurs hyperparam\^etres architecturaux influencent les performances. Les enseignements principaux sont les suivants :

- **Le MLP est fondamentalement inadapt\^e aux images** en raison de l'explosion des param\^etres (400K+ pour une seule couche), de la perte de structure spatiale lors du flatten, et de l'absence d'invariance aux translations.
- **Le CNN encode un biais inductif adapt\^e** (localit\^e, stationnarit\^e, compositivit\^e) qui lui permet d'atteindre de meilleures performances avec $\sim 9\times$ moins de param\^etres que le MLP.
- **Le Max-Pooling surpasse l'Average-Pooling** pour la classification car il s\^electionne la r\^eponse dominante, d\^etectant la pr\^esence d'un motif ind\^ependamment de sa position exacte dans la fen\^etre.
- **Le same-padding ($p = (k-1)/2$)** pr\^eserve les dimensions spatiales, permettant des architectures plus profondes et une meilleure exploitation de l'information aux bords.
- **Batch Normalization + Dropout + Xavier** constituent une trilogie de stabilisation qui am\^eliore la convergence, r\^eduit l'overfitting et diminue la sensibilit\^e aux hyperparam\^etres.
- **Les filtres appris spontan\^ement ressemblent aux d\^etecteurs de V1** (contours, orientations), validant empiriquement la pertinence du biais inductif des CNN pour les images naturelles.

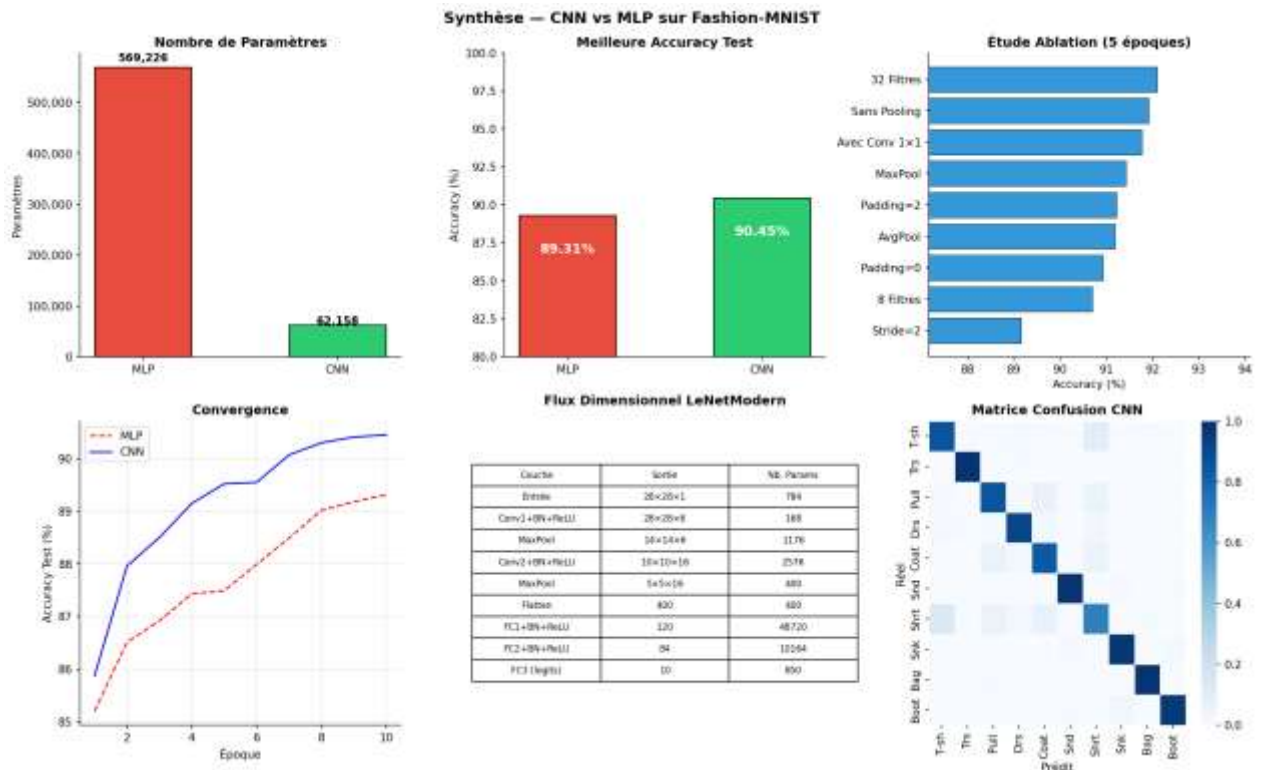


Figure 12 : Synthèse visuelle finale (6 sous-graphiques)

Cette planche de synthèse regroupe les résultats clés du chapitre : nombre de paramètres, accuracy finale, étude d'ablation, courbes de convergence, flux dimensionnel du CNN et matrice de confusion, offrant une vue d'ensemble comparative entre MLP et CNN sur Fashion-MNIST.

Performance finale : LeNetModern atteint ~91% d'accuracy sur Fashion-MNIST avec seulement ~62 000 paramètres, contre ~89% pour le MLP avec ~549 000 paramètres. Gain : +2 points avec 9× moins de paramètres. L'efficacité paramétrique du CNN (~1 450%/M params) est ~9× supérieure à celle du MLP (~160%/M params) sur cette tâche.



3. Chapitre 3 : Modélisation des données séquentielles avec les RNN

3.1. Introduction et Positionnement

Les architectures récurrentes constituent le socle historique de la modélisation de séquences en apprentissage profond. Contrairement aux réseaux denses (MLP) qui traitent chaque vecteur d'entrée de façon indépendante, et aux réseaux convolutifs (CNN) qui exploitent des invariances spatiales locales, les RNN sont conçus pour capturer des dépendances temporelles ou séquentielles entre des éléments ordonnés. Les données concernées incluent le texte naturel, la parole, les séries temporelles financières, les trajectoires de mouvement et toute séquence où l'ordre des observations porte une signification causale.

La difficulté centrale est la suivante : le sens d'un token (mot, phonème, symbole) dépend du contexte passé — parfois de l'élément immédiatement précédent, parfois d'un antécédent éloigné de plusieurs dizaines de positions. L'enjeu est donc de représenter efficacement ce passé sous la forme d'un état interne compact mais expressif.

Cette synthèse retrace la progression complète qui va de la formalisation probabiliste des séquences jusqu'aux architectures encodeur-décodeur pour la traduction automatique :

- Modélisation probabiliste d'une séquence : modèles de langage, perplexité
- Architecture récurrente de base (RNN vanilla) et son apprentissage par rétropropagation à travers le temps (BPTT)
- Architectures avancées à portes : LSTM (Long Short-Term Memory) et GRU (Gated Recurrent Unit)
- Variantes architecturales : RNN profonds, bidirectionnels, dropout récurrent
- Préparation des données séquentielles pour la traduction automatique (Tatoeba fra-eng)
- Architecture encodeur-décodeur et modèle Seq2Seq avec Teacher Forcing
- Stratégies de décodage : décodage glouton et Beam Search
- Évaluation quantitative : Perplexité et score BLEU

3.2. Modèles de Langage — Fondements Probabilistes

3.2.1. Objectif probabiliste

Un modèle de langage (Language Model, LM) est un modèle probabiliste défini sur des séquences de tokens. Il attribue une probabilité à toute séquence $(w_1, w_2, \dots, w_T)$ appartenant à V^* (l'ensemble de toutes les séquences possibles sur le vocabulaire V). Par application de la règle de chaîne des probabilités conditionnelles, cette probabilité jointe se décompose sans approximation de la façon suivante :

$$P(w_1, w_2, \dots, w_T) = \prod_t P(w_t | w_1, \dots, w_{t-1}) = \prod_t P(w_t | w_{<t})$$

Cette factorisation est exacte : elle ne repose sur aucune hypothèse d'indépendance ou de Markov. Elle révèle l'intuition fondatrice : pour modéliser une séquence entière, il suffit de savoir prédire le prochain token à partir de tout le contexte passé. C'est cette capacité de prédiction conditionnelle que les architectures

récurrentes cherchent à approximer. Toute la difficulté réside dans la représentation compacte et expressive de ce contexte $w_{<t}$ sous la forme d'un état interne de dimension finie.

3.2.2. Log-probabilités et Negative Log-Likelihood

En pratique, les probabilités individuelles sont très petites (de l'ordre de 10^{-5} ou moins), ce qui rend leur multiplication numériquement instable — un phénomène connu sous le nom d'underflow numérique. On travaille donc systématiquement en espace logarithmique. La log-probabilité d'une séquence est :

$$\log P(w) = \sum_t \log P(w_t | w_{<t})$$

L'objectif d'entraînement est la maximisation de la vraisemblance (Maximum Likelihood Estimation, MLE) sur le corpus d'entraînement, ce qui est équivalent à la minimisation de la Cross-Entropie Négative (Negative Log-Likelihood, NLL) :

$$L(\theta) = - (1/N) \sum_n \sum_t \log P_{\theta}(w_t^n | w_{<t}^n)$$

Cette fonction de perte est différentiable par rapport aux paramètres θ du réseau et peut donc être minimisée par descente de gradient stochastique. Elle pénalise le modèle chaque fois qu'il assigne une faible probabilité au vrai token suivant.

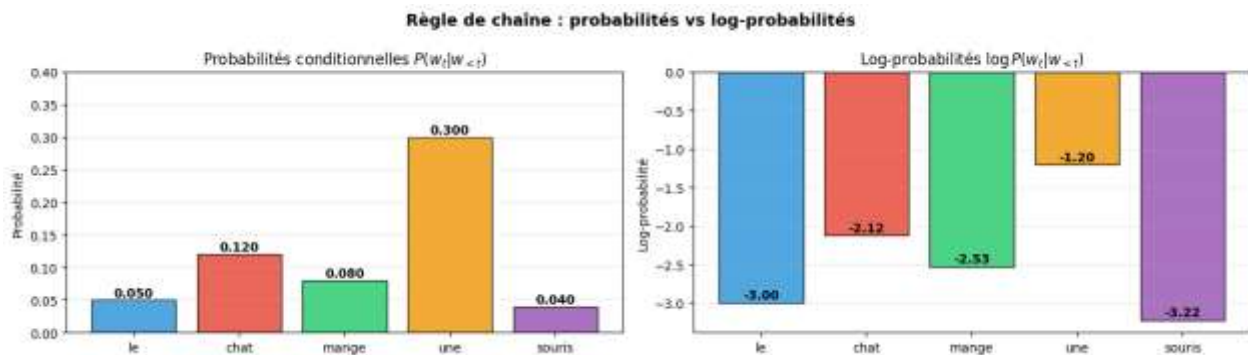


Figure 13 : Règle de chaîne : probabilités vs log-probabilités

Cette figure illustre numériquement la décomposition de la règle de chaîne sur l'exemple "le chat mange une souris", comparant les probabilités conditionnelles brutes à leurs log-transformées. Elle confirme visuellement que la probabilité jointe devient rapidement négligeable, justifiant le recours systématique à l'espace logarithmique pour l'entraînement.

3.2.3. Perplexité — Métrique Standard

La perplexité (PPL) est la métrique canonique pour évaluer un modèle de langage. Elle est définie comme l'exponentielle de la NLL moyenne par token :

$$PPL = \exp(- (1/T) \sum_t \log P(w_t | w_{<t})) = \exp(L)$$

Intuitivement, la perplexité est le nombre effectif de choix équiprobables que le modèle perçoit à chaque position. Un modèle parfait (certitude absolue) aurait $PPL = 1$; un modèle uniforme sur le vocabulaire

(n'ayant rien appris) aurait $PPL = |V|$. La perplexité est une métrique inversée : plus elle est basse, meilleur est le modèle.

Perplexité	Interprétation	Exemple pratique
$PPL = 1$	Modèle parfait — certitude absolue	Impossible en pratique
$PPL = 10$	Hésite entre ~ 10 tokens — très bon	LSTM sur grand corpus
$PPL = 50\text{--}100$	Modèle NLP standard	Résultat habituel Tatoeba
$PPL = V $	Modèle uniforme — n'a rien appris	Avant tout entraînement

Tableau 23 : Tableau des perplexités et interprétations

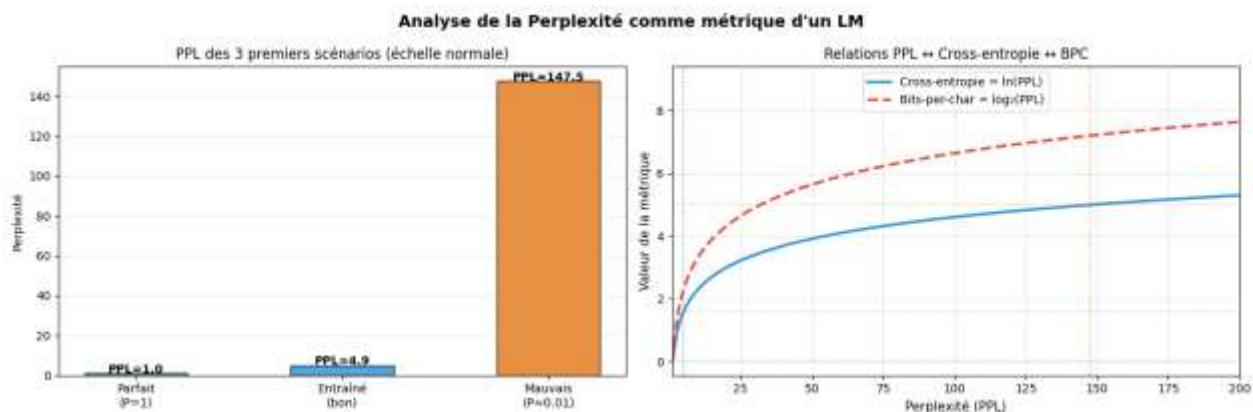


Figure 14 : Analyse de la perplexité selon différents scénarios

Cette figure compare la perplexité obtenue pour un modèle parfait, un modèle entraîné et un mauvais modèle, ainsi que la relation mathématique entre perplexité, cross-entropie et bits-per-character. Elle met en évidence l'écart d'ordre de grandeur entre un modèle performant et un modèle proche du hasard.

3.2.4. Approche N-grammes versus Modèles Neuronaux

L'approche classique des n-grammes approxime $P(w_t | w_{<t})$ par $P(w_t | w_{t-n+1}, \dots, w_{t-1})$, limitant le contexte aux $n-1$ tokens précédents. Cette approximation de Markov souffre de deux limitations fondamentales : la sparsité des données (les n-grammes rares sont mal estimés) et l'incapacité à capturer des dépendances s'étendant au-delà de la fenêtre de n tokens. Les RNN, LSTM et GRU n'ont pas cette limitation théorique : ils traitent l'intégralité de l'histoire via leur état caché h_t , qui est mis à jour récursivement à chaque nouveau token.

3.3. Représentation Vectorielle des Tokens — Embeddings

Pour qu'un réseau neuronal puisse traiter du texte, chaque token discret doit être représenté par un vecteur numérique. Deux approches s'affrontent historiquement :

3.3.1. Représentation one-hot

La représentation one-hot associe à chaque token w_t un vecteur de dimension $|V|$ contenant un seul 1 à la position correspondante et des 0 partout ailleurs. Cette approche souffre de deux défauts rédhibitoires : (1) une dimensionnalité catastrophique pour les grands vocabulaires ($|V|$ peut atteindre 50 000 à 100 000 tokens), et (2) l'absence totale de sémantique — tous les tokens sont équidistants les uns des autres dans cet espace, ce qui empêche le modèle d'exploiter les similarités sémantiques entre mots.

3.3.2. Embeddings denses appris

La solution moderne est la couche d'embedding : une matrice $E \in \mathbb{R}^{|V| \times d}$ apprise par rétropropagation conjointement avec le reste du réseau. Chaque token est représenté par une ligne de cette matrice — un vecteur dense de dimension d (typiquement 64 à 512) :

$$\mathbf{x}_t = E[\mathbf{w}_t] \in \mathbb{R}^d \text{ — ligne de la matrice } E \text{ indexée par } \mathbf{w}_t$$

Les embeddings appris possèdent une propriété remarquable : les tokens sémantiquement proches sont représentés par des vecteurs proches dans l'espace d'embedding. Cette propriété émerge naturellement de l'entraînement, sans supervision explicite. Le token spécial PAD (rembourrage) est conventionnellement assigné à l'index 0 avec `padding_idx=0`, ce qui force son embedding à rester un vecteur nul (gradient nul, aucune mise à jour).

```
# PyTorch — Couche d'embedding
embed = nn.Embedding(vocab_size, embed_dim=128, padding_idx=0)

# x : (B, T) — indices des tokens (entiers)
# e = embed(x) : (B, T, 128) — vecteurs denses appris
```

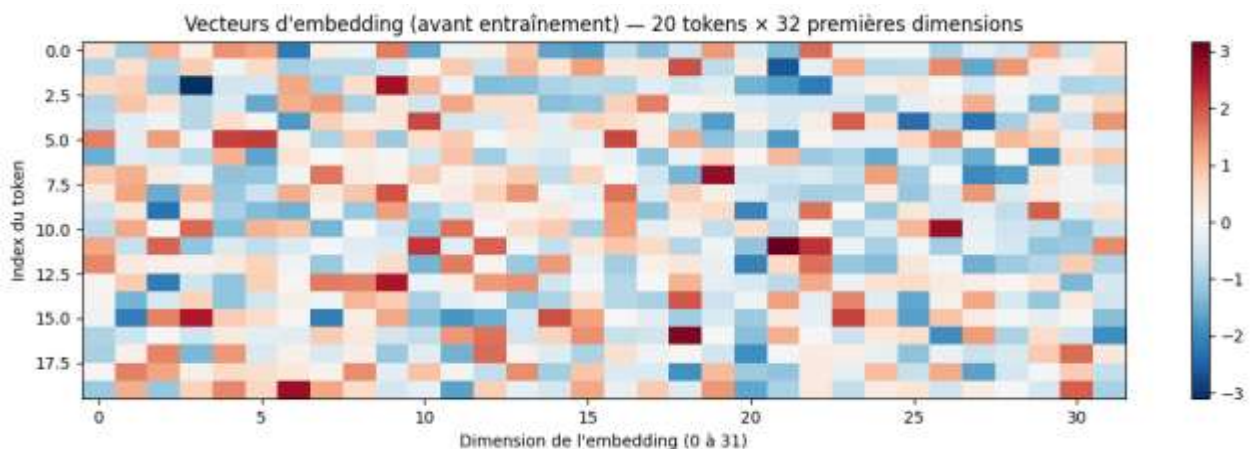


Figure 15 : Visualisation des vecteurs d'embedding avant entraînement

Cette carte de chaleur représente les 32 premières dimensions des vecteurs d'embedding pour 20 tokens avant tout entraînement, illustrant leur caractère initialement aléatoire. Ces vecteurs évolueront pendant l'apprentissage pour capturer des relations sémantiques et syntaxiques entre les mots.

3.4. Réseau Neuronnel Récurrent Vanilla (RNN)

3.4.1. Idée centrale — La mémoire récurrente

Un réseau dense (MLP) traite chaque entrée indépendamment : il est incapable de capturer les dépendances temporelles entre les éléments d'une séquence. Le RNN résout ce problème en maintenant un état caché h_t — un vecteur de dimension H — qui joue le rôle de mémoire compressée de tout le passé de la séquence. Cet état est mis à jour à chaque pas de temps en combinant l'entrée courante x_t et l'état précédent h_{t-1} .

3.4.2. Équations fondamentales

À chaque pas de temps t , le RNN calcule deux quantités :

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b_h)$$

$$y_t = W_{hy} \cdot h_t + b_y \text{ — logits de prédiction vers le vocabulaire}$$

Symbole	Dimensions	Rôle
x_t	$\mathbb{R}^d$	Vecteur d'embedding du token w_t
h_t	$\mathbb{R}^H$	État caché — mémoire compressée du passé
W_{hh}	$\mathbb{R}^{H \times H}$	Matrice de transition caché→caché (cœur de la récurrence)
W_{xh}	$\mathbb{R}^{H \times d}$	Matrice d'entrée→caché
h_0	$\mathbb{R}^H$	État initial (vecteur nul ou appris)

Tableau 24 : Symboles et dimensions du RNN vanilla

3.4.3. Partage des poids dans le temps

Un aspect fondamental du RNN est que les matrices W_{hh} , W_{xh} et W_{hy} sont partagées à tous les pas de temps : le même réseau est appliqué récursivement sur toute la séquence. Ce partage de poids permet au RNN de traiter des séquences de longueur arbitraire avec un nombre fixe de paramètres, et l'oblige à développer des représentations générales et invariantes dans le temps.

3.4.4. Décompte des paramètres

Pour une couche RNN avec H neurones cachés et des embeddings de dimension d , le nombre total de paramètres est :

$$\text{Params RNN} = H \times (d + H + 1)$$

Exemple numérique : avec $H = 256$ et $d = 128$, on obtient $256 \times (128 + 256 + 1) = 98\,560$ paramètres par couche — un modèle relativement compact.

3.4.5. Implémentation PyTorch

```
rnn = nn.RNN(input_size=128, hidden_size=256, num_layers=2,
             batch_first=True, dropout=0.3)

# X : (B, T, d) — séquence d'embeddings
# H0 : (L, B, H) — état initial (L=nb couches)
```

```
output, hidden = rnn(X, H0)
# output : (B, T, H) — sorties à chaque pas de temps
# hidden : (L, B, H) — état caché final
```

3.5. Apprentissage — BPTT et Stabilisation des Gradients

3.5.1. Rétropropagation à travers le temps (BPTT)

L'algorithme de Backpropagation Through Time (BPTT) est l'extension directe de la rétropropagation standard aux réseaux récurrents. Il consiste à dérouler le réseau récurrent dans le temps — en créant une copie du réseau pour chaque pas de temps — puis à appliquer la règle de la chaîne pour calculer les gradients. La perte totale est la somme des pertes à chaque position :

$$L = \sum_t L_t$$

Le gradient de la perte par rapport à l'état caché k doit traverser tous les pas de temps de k jusqu'à T :

$$\partial L / \partial \mathbf{h}_k = \sum_{t \geq k} (\partial L_t / \partial \mathbf{h}_t) \times \prod_{i=k}^{t-1} (\partial \mathbf{h}_{i+1} / \partial \mathbf{h}_i)$$

Chaque terme $\partial \mathbf{h}_{i+1} / \partial \mathbf{h}_i$ implique une multiplication par la matrice W_{hh} (pondérée par la dérivée de $\tanh$). C'est ce produit répété de matrices qui engendre les problèmes de gradient.

3.5.2. Problèmes de gradient — Vanishing et Exploding

L'analyse spectrale du produit de matrices révèle deux phénomènes critiques, directement liés à la valeur propre maximale $\lambda_{\max}$ de W_{hh} :

- Gradient évanescent (Vanishing Gradient) : si $\lambda_{\max} < 1$, le produit de matrices tend vers zéro exponentiellement avec la profondeur temporelle. Le modèle n'apprend pas les dépendances longues — les gradients sont trop faibles pour modifier les poids correspondant aux tokens éloignés.
- Gradient explosif (Exploding Gradient) : si $\lambda_{\max} > 1$, le produit devient exponentiellement grand. Les mises à jour sont gigantesques et l'entraînement diverge — les poids oscillent ou deviennent des NaN.

Ces deux pathologies sont intrinsèques à l'architecture RNN vanilla et constituent la motivation principale pour le développement du LSTM et du GRU.

3.5.3. Gradient Clipping — Solution contre l'Explosion

La solution standard contre l'explosion de gradient est le clipping par norme : si la norme L2 du gradient dépasse un seuil λ , le vecteur de gradient est renormalisé à la norme λ . La direction du gradient est préservée, seule sa magnitude est bornée :

$$\mathbf{g} \leftarrow \mathbf{g} \times \min(1, \lambda / \|\mathbf{g}\|_2) \text{ si } \|\mathbf{g}\|_2 > \lambda$$

```
Après loss.backward(), avant optimizer.step()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
optimizer.step()
```

Valeur recommandée : $\lambda \in [1, 5]$ pour les RNN (Sutskever et al., 2014)

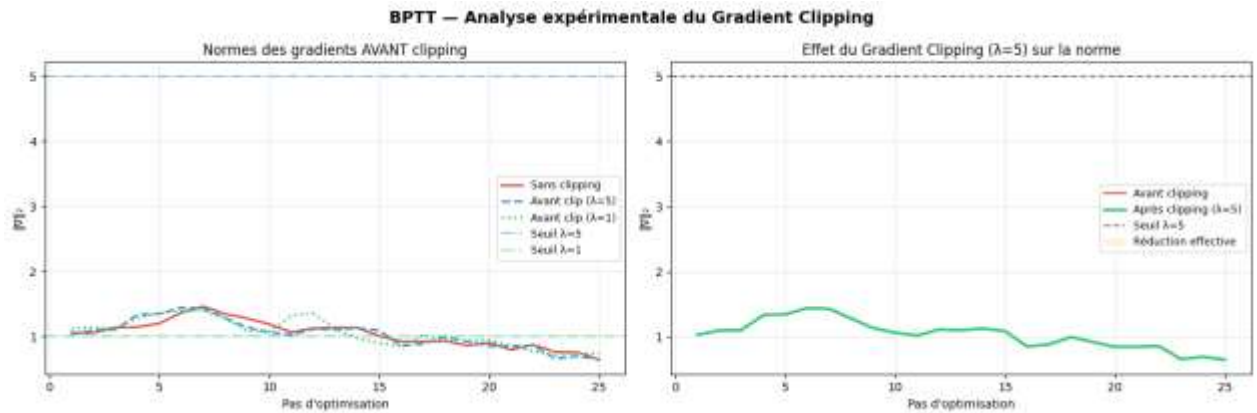


Figure 16 : Effet du Gradient Clipping sur la norme des gradients

Cette figure montre l'évolution de la norme des gradients avant et après application du clipping pour différents seuils λ . On observe que sans clipping, certains pics de gradient dépassent largement le seuil de stabilité, tandis que le clipping à $\lambda=5$ contient efficacement ces explosions sans pénaliser excessivement l'apprentissage.

3.5.4. BPTT Tronquée

Une variante pratique est la BPTT tronquée (Truncated BPTT) : on limite la profondeur de la rétropropagation à τ pas de temps. Les états cachés sont propagés en avant sur toute la séquence, mais les gradients ne sont rétropropagés que sur τ pas. Cette approche réduit le coût mémoire de $O(T)$ à $O(\tau)$ et atténue mécaniquement le vanishing/exploding, au prix de ne pas capturer les dépendances s'étendant sur plus de τ positions.

3.6. LSTM — Long Short-Term Memory

3.6.1. Motivation et Innovation

Le LSTM (Hochreiter & Schmidhuber, 1997) est la réponse architecturale directe au problème de mémoire du RNN vanilla. Il introduit deux innovations structurelles majeures : (1) un état de cellule c_t , qualifié d'autoroute d'information car il permet au gradient de s'écouler sans dégradation sur de longues distances, et (2) un mécanisme de portes (gates) — des fonctions sigmoïdes apprises — qui contrôlent sélectivement quelles informations sont retenues, écrites ou exposées à chaque pas de temps.

3.6.2. Équations Complètes du LSTM

Notons $[h_{t-1}, x_t]$ la concaténation des vecteurs h_{t-1} et x_t . Le LSTM calcule à chaque pas de temps six quantités selon les équations suivantes :

Porte	Équation	Rôle fonctionnel
Oubli f_t	$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$	Quelle fraction de c_{t-1} conserver ? (0 = effacer, 1 = garder)

Entrée i_t	$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$	Quelle fraction du candidat écrire dans c_t ?
Candidat $\tilde{c}_t$	$\tilde{c}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$	Nouvelle information potentielle à intégrer
Cellule c_t	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	Mémoire longue terme (mise à jour additive)
Sortie o_t	$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$	Quelle fraction de c_t exposer à l'état caché ?
Caché h_t	$h_t = o_t \odot \tanh(c_t)$	Sortie du LSTM — représentation du contexte courant

Tableau 25 : Équations complètes du LSTM (6 portes)

3.6.3. Pourquoi c_t Résout le Gradient Évanescent

La clé du LSTM réside dans la nature de la mise à jour de la cellule : $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. Cette opération est additive, contrairement à la multiplication matricielle du RNN vanilla. Le gradient de c_t par rapport à c_k ($k < t$) est :

$$\partial c_t / \partial c_k = \prod_{i=k}^{t-1} f_i \text{ — produit de scalaires, pas de matrices}$$

Si le LSTM apprend $f_t \approx 1$ (porte d'oubli quasi-ouverte), les gradients traversent toute la séquence presque sans atténuation. Le flux de gradient est ainsi préservé sur de longues distances, ce qui permet au LSTM de capturer des dépendances de plusieurs dizaines ou centaines de pas de temps.

3.6.4. Décompte des Paramètres

Le LSTM comporte 4 matrices de poids (une par porte et pour le candidat), ce qui quadruple le nombre de paramètres par rapport au RNN :

$$\text{Params LSTM} = 4 \times H \times (d + H + 1) = 4 \times \text{Params RNN}$$

Exemple : avec $H = 256$ et $d = 128$, le LSTM atteint $4 \times 98\,560 = 394\,240$ paramètres par couche. Ce coût accru est la contrepartie de la capacité de mémoire supérieure.

3.6.5. Implémentation PyTorch

```
lstm = nn.LSTM(input_size=128, hidden_size=256, num_layers=2,
               batch_first=True, dropout=0.3)
```

L'état caché du LSTM est un TUPLE (h, c)
 output, (h_n, c_n) = lstm(X, (h0, c0))
 output : (B, T, H)
 h_n : (L, B, H) — état caché final
 c_n : (L, B, H) — état de cellule final

3.7. GRU — Gated Recurrent Unit

3.7.1. Principe de Simplification

Le GRU (Cho et al., 2014) reformule le LSTM en une architecture plus économique : il supprime l'état de cellule séparé et fusionne les portes d'oubli et d'entrée en une unique porte de mise à jour z_t . La mémoire est portée directement par l'état caché h_t . Cette simplification réduit le nombre de paramètres de 25 % tout en conservant des performances comparables au LSTM sur la majorité des tâches pratiques.

3.7.2. Équations du GRU

Composante	Équation	Rôle
Mise à jour z_t	$z_t = \sigma(W_z [h_{t-1}, x_t] + b_z)$	Interpolation entre ancien et nouveau (0=garder, 1=réécrire)
Réinitialisation r_t	$r_t = \sigma(W_r [h_{t-1}, x_t] + b_r)$	Quelle fraction du passé utiliser pour le candidat ?
Candidat $\tilde{h}_t$	$\tilde{h}_t = \tanh(W_h [r_t \odot h_{t-1}, x_t] + b_h)$	Nouvelle information potentielle
État h_t	$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$	Interpolation linéaire passé / nouveau

Tableau 26 : Équations du GRU (4 composantes)

L'équation de mise à jour $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$ réalise une interpolation convexe : si $z_t \rightarrow 0$, l'état est copié intégralement du passé ; si $z_t \rightarrow 1$, il est entièrement réécrit par le candidat. Cette interpolation est fonctionnellement analogue à la mise à jour additive du LSTM, avec la contrainte implicite $i_t = 1 - f_t$.

3.7.3. Comparaison Exhaustive LSTM / GRU

Critère	LSTM	GRU
Portes	3 (oubli, entrée, sortie) + candidat	2 (mise à jour + réinitialisation)
État interne	Double : $h_t + c_t$	Simple : h_t uniquement
Paramètres	$4 \times H(d+H+1)$	$3 \times H(d+H+1)$ — 25 % de moins
Vitesse d'entraînement	Plus lente	Plus rapide
Dépendances très longues	Légèrement supérieur	Comparable
Recommandation	Séquences très longues (>100 tokens)	Bon compromis polyvalent

Tableau 27 : Comparaison exhaustive LSTM vs GRU

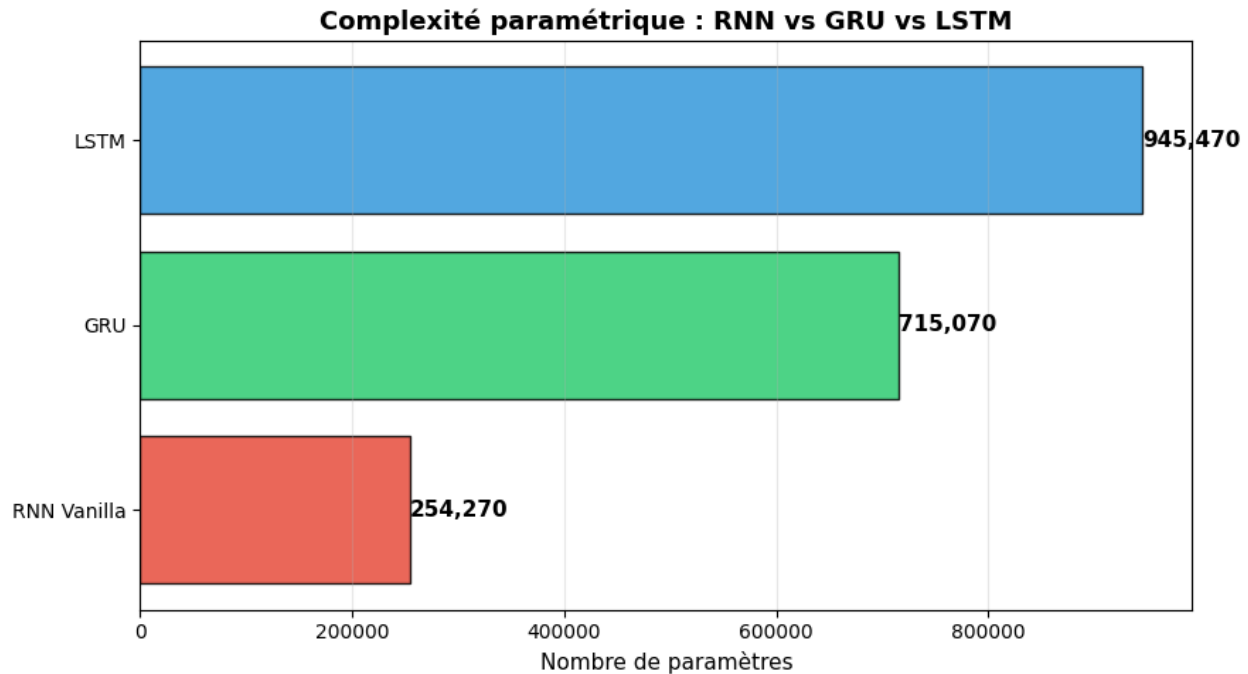


Figure 17 : Comparaison de la complexité paramétrique RNN vs GRU vs LSTM

Ce graphique à barres horizontales confirme visuellement le rapport attendu entre les trois architectures : le GRU se positionne entre le RNN vanilla et le LSTM en nombre de paramètres, avec environ 25 % de paramètres en moins que le LSTM pour une capacité de mémoire comparable.

3.7.4. Implémentation PyTorch

```
gru = nn.GRU(input_size=128, hidden_size=256, num_layers=2,
             batch_first=True, dropout=0.3)
```

L'état caché du GRU est un tenseur simple (pas un tuple)

output, h_n = gru(X, h0)

output : (B, T, H)

h_n : (L, B, H) — état caché final

3.8. Variantes Architecturales

3.8.1. RNN Profonds — Empilement de Couches

Empiler L couches récurrentes crée un RNN profond : la sortie de la couche ℓ au pas t devient l'entrée de la couche $\ell+1$. Formellement :

$$\mathbf{h}_t^{(\ell)} = f^{\ell}(\mathbf{h}_t^{(\ell-1)}, \mathbf{h}_{t-1}^{(\ell)}) \text{ pour } \ell \geq 2$$

L'empilement crée une hiérarchie de représentations : la couche basse capture des motifs locaux et syntaxiques, les couches supérieures extraient des représentations plus abstraites et sémantiques. En pratique, $L = 2$ à 4 couches suffisent pour la plupart des tâches de NLP. Au-delà, l'entraînement devient difficile sans techniques de résidualité.

```
4 couches empilées
rnn = nn.RNN(input_size=128, hidden_size=256, num_layers=4,
             batch_first=True, dropout=0.3)
```

3.8.2. Dropout Récurrent

Le dropout est appliqué entre les couches récurrentes — sur les connexions verticales — et non sur les connexions horizontales intra-couche (récurrences temporelles). Cette distinction est fondamentale : appliquer du dropout sur les récurrences perturberait le flux d'information temporelle et dégraderait les performances. Le taux recommandé est $p \in [0.2, 0.4]$. Un taux trop élevé ($p > 0.5$) provoque une perte d'information mémorielle excessive et nuit à l'apprentissage des dépendances longues.

3.8.3. RNN Bidirectionnels

Un RNN bidirectionnel traite la séquence en parallèle dans les deux directions temporelles — de gauche à droite et de droite à gauche — et concatène les états cachés correspondants à chaque position :

```
h_t = [→h_t ; ←h_t] — concaténation avant et arrière, dimension 2H
```

Cette approche est précieuse pour les tâches d'analyse qui bénéficient du contexte futur : étiquetage de séquences (NER, POS tagging), classification de phrase, encodage pour la traduction. Elle est en revanche inutilisable pour les tâches auto-régressives (génération de texte, décodage en traduction automatique) : à l'inférence, le futur n'est pas encore disponible.

```
rnn = nn.LSTM(input_size=128, hidden_size=256,
             bidirectional=True, batch_first=True)
sortie : (B, T, 2*H) — 2 directions concaténées
```

3.9. Préparation des Données — Tatoeba fra-eng

3.9.1. Description du Dataset

Le corpus Tatoeba fra-eng est un ensemble de paires de phrases bilingues anglais ↔ français. Le corpus complet contient environ 190 000 paires ; nous en utilisons 8 000 après filtrage sur une longueur maximale de 12 tokens par phrase. Les phrases sont courtes et couvrent des sujets quotidiens, avec un vocabulaire relativement limité. Ce dataset constitue un banc d'essai idéal pour expérimenter les architectures Seq2Seq à faible coût computationnel.

3.9.2. Pipeline de Prétraitement

Le pipeline de prétraitement comporte six étapes successives, chacune indispensable à la qualité de l'entraînement :

- Normalisation : conversion unicode → ASCII, mise en minuscules, insertion d'espaces autour de la ponctuation pour garantir une tokenisation cohérente
- Tokenisation : découpage par espace (word-level) — approche simple et suffisante pour ce corpus
- Construction du vocabulaire : dictionnaire bidirectionnel token ↔ index, construit séparément pour le français (source) et l'anglais (cible)

- Tokens spéciaux : quatre tokens réservés avec des rôles précis
- Encodage : conversion de chaque séquence de tokens en liste d'entiers, suivie de l'ajout du token EOS
- Padding et masquage : alignement des séquences d'un même batch sur la longueur maximale, avec création d'un masque booléen

Token	Index	Rôle dans l'architecture
<PAD>	0	Rembourrage — aligne les séquences de longueurs différentes dans le batch
<SOS>	1	Start of Sequence — signal de démarrage pour le décodeur
<EOS>	2	End of Sequence — signal d'arrêt de la génération
<UNK>	3	Unknown — remplace les tokens absents du vocabulaire (OOV)

Tableau 28 : Tokens spéciaux utilisés dans l'architecture du modèle

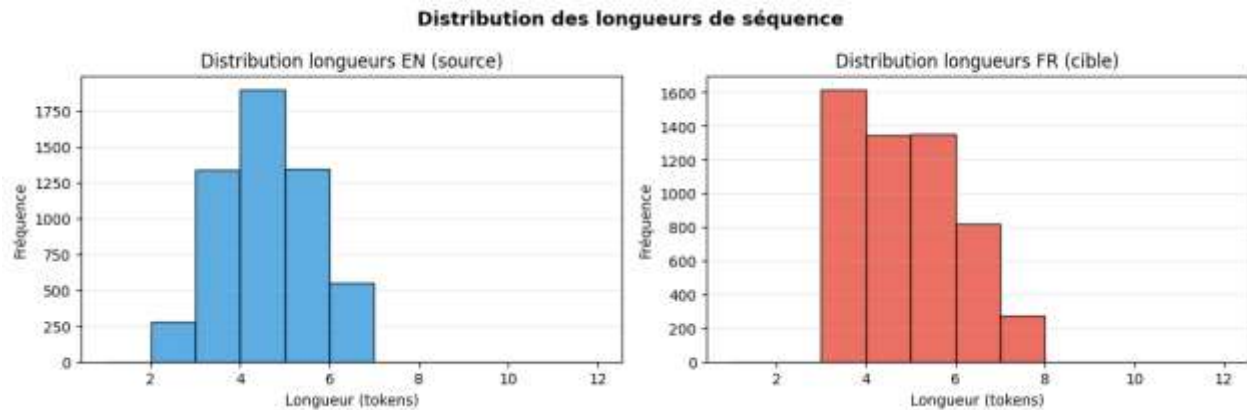


Figure 18 : Distribution des longueurs de séquences (EN/FR)

Ces histogrammes présentent la distribution des longueurs de phrases en anglais et en français après filtrage à 12 tokens maximum, confirmant la relative homogénéité des longueurs entre les deux langues et justifiant le choix d'une longueur maximale modérée pour limiter le padding excessif.

3.9.3. Padding, Masquage et Perte Masquée

Les phrases d'un même mini-lot ont des longueurs différentes. On les aligne sur la longueur maximale du batch en ajoutant des tokens <PAD> à droite. Le masque de padding ($m_t = 1$ si position valide, 0 si PAD) est ensuite utilisé pour ignorer les positions PAD dans le calcul de la perte :

$$L_{\text{masquée}} = (\sum_t m_t \times \ell_t) / \sum_t m_t$$

Sans masquage, le modèle gaspillerait de la capacité à prédire des tokens PAD, ce qui dégraderait les performances et biaiserait les métriques. Le masquage garantit que seules les positions porteuses d'information réelle contribuent au gradient.

```

Collation avec padding automatique
src_padded = nn.utils.rnn.pad_sequence(src_batch, batch_first=True, padding_value=0)
src_mask = (src_padded != 0) True = token réel, False = PAD

```

3.10. Architecture Seq2Seq — Encodeur–Décodeur

3.10.1. Limitation du Modèle de Langage Simple

Un modèle de langage modélise la distribution marginale $P(w_t | w_{<t})$ sur une unique séquence. La traduction automatique est un problème fondamentalement différent : il faut modéliser la distribution conditionnelle $P(y_1, \dots, y_M | x_1, \dots, x_N)$ — une distribution sur la séquence cible (français) conditionnée à la séquence source (anglais). Les deux séquences ont des longueurs différentes ($N \neq M$ en général) et des vocabulaires entièrement distincts. Un LM simple ne peut pas résoudre ce problème.

3.10.2. Architecture Encodeur–Décodeur

Le modèle Seq2Seq (Sutskever, Vinyals & Le, 2014) décompose le problème en deux phases séparées mais interdépendantes :

- Phase 1 — Encodage : le réseau encodeur (un GRU ou LSTM) lit la séquence source $x = (x_1, \dots, x_N)$ token par token et produit un vecteur de contexte $c = h_N^{\text{enc}}$ (l'état caché final, comprimant toute la séquence source en un vecteur de dimension H)
- Phase 2 — Décodage : le réseau décodeur génère la séquence cible $y = (y_1, \dots, y_M)$ token par token, initialisé avec c et conditionné à chaque pas sur sa propre sortie précédente

La décomposition probabiliste sous-jacente est :

$$P(y | x) = \prod_t P(y_t | y_{<t}, x)$$

Les équations du Seq2Seq avec un décodeur GRU sont :

$$\begin{aligned}
c &= h_N^{\text{enc}} \text{ (vecteur de contexte — état final de l'encodeur)} \\
s_0 &= c, \quad s_t = \text{GRU}_{\text{dec}}(y_{t-1}, s_{t-1}), \quad P(y_t | \dots) = \text{softmax}(W \cdot s_t)
\end{aligned}$$

3.10.3. Teacher Forcing

Pendant l'entraînement, deux stratégies s'offrent pour choisir l'entrée du décodeur au pas t :

Stratégie	Entrée du décodeur au pas t	Avantage	Inconvénient
Teacher Forcing	Vrai token y_{t-1}^* (gold)	Convergence rapide et stable	Exposure Bias
Auto-régressif	$\hat{y}_{t-1} = \arg\max P(\cdot)$	Proche de l'inférence réelle	Convergence lente, instable

Tableau 29 : Teacher Forcing vs Auto-régressif

L'Exposure Bias est la pathologie centrale du Teacher Forcing : à l'entraînement, le décodeur est toujours alimenté par la vérité (tokens gold), mais à l'inférence il doit utiliser ses propres prédictions — potentiellement erronées. Cette divergence train/inférence peut provoquer une accumulation d'erreurs. La

solution recommandée est le curriculum progressif : le ratio de Teacher Forcing commence à 1.0 (100 % gold) et décroît vers 0.2 au cours de l'entraînement.

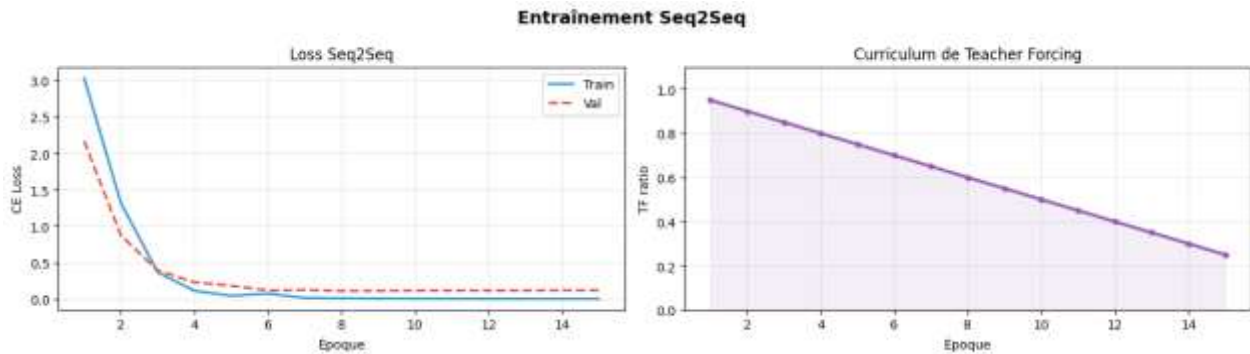


Figure 19 : Entraînement du modèle Seq2Seq et curriculum de Teacher Forcing

Cette figure illustre conjointement la convergence de la loss d'entraînement et de validation du modèle Seq2Seq, ainsi que la décroissance programmée du ratio de Teacher Forcing au fil des époques, passant de 1.0 à 0.2 selon le curriculum décrit précédemment.

3.10.4. Bottleneck du Vecteur de Contexte

La limitation fondamentale du Seq2Seq classique est le bottleneck du vecteur de contexte : toute l'information de la séquence source (potentiellement longue) doit être comprimée en un unique vecteur $c \in \mathbb{R}^H$. Pour les phrases longues, cette compression est destructive — des informations cruciales sont perdues. La solution est le mécanisme d'attention (Bahdanau et al., 2015), qui permet au décodeur d'accéder sélectivement à tous les états cachés de l'encodeur à chaque pas de décodage, plutôt que de dépendre d'un unique résumé fixe.

3.10.5. Implémentation PyTorch

```
class Encoder(nn.Module):
    def __init__(self, input_dim, embed_dim, hidden_dim, n_layers):
        super().__init__()
        self.embedding = nn.Embedding(input_dim, embed_dim, padding_idx=0)
        self.gru = nn.GRU(embed_dim, hidden_dim, n_layers, batch_first=True)

    def forward(self, src): # src : (B, src_len)
        emb = self.embedding(src) # (B, src_len, d)
        outputs, hidden = self.gru(emb) # (B, src_len, H), (L, B, H)
        return outputs, hidden

class Decoder(nn.Module):
    def forward(self, tgt_token, hidden): # tgt_token : (B,)
        emb = self.embedding(tgt_token.unsqueeze(1)) # (B, 1, d)
        output, hidden = self.gru(emb, hidden) # (B, 1, H)
        logits = self.fc_out(output.squeeze(1)) # (B, V_tgt)
        return logits, hidden
```

```

Boucle Seq2Seq avec Teacher Forcing
_, hidden = encoder(src)          # c = h_T
dec_input = tgt[:, 0]             # <SOS>
for t in range(tgt_len - 1):
    logits, hidden = decoder(dec_input, hidden)
    use_teacher = random.random() < tf_ratio
    dec_input = tgt[:, t+1] if use_teacher else logits.argmax(dim=-1)

```

3.11. Stratégies de Décodage à l'Inférence

3.11.1. Formulation du Problème

À l'inférence, le problème de décodage est la recherche de la séquence cible la plus probable conditionnellement à la source :

$$y^* = \operatorname{argmax}_y P(y \mid x) = \operatorname{argmax}_y \sum_t \log P(y_t \mid y_{<t}, x)$$

L'espace de recherche est de taille $|V|^M$ — exponentiellement grand — et ne peut pas être parcouru exhaustivement. On utilise des heuristiques de recherche approchée.

3.11.2. Décodage Glouton (Greedy Decoding)

Le décodage glouton sélectionne à chaque pas le token le plus probable selon la distribution de sortie courante :

$$y_t^* = \operatorname{argmax}_w P(w \mid y_{<t^*}, x)$$

Sa complexité est $O(T \times |V|)$, ce qui le rend très rapide. Cependant, il est sous-optimal globalement : un excellent premier choix peut conduire à une impasse — une mauvaise séquence partielle dont aucun complément n'est de bonne qualité. Les erreurs sont non récupérables une fois commises.

3.11.3. Beam Search

Le Beam Search généralise le décodage glouton en maintenant les k meilleures hypothèses partielles à chaque étape (k est appelé la largeur du faisceau). Les hypothèses sont scorées par leur log-probabilité cumulée :

$$\operatorname{score}(y_1, \dots, y_t) = \sum_i \log P(y_i \mid y_{<i}, x)$$

Algorithme du Beam Search :

- Initialiser avec k copies de <SOS>
- À chaque pas, étendre chaque hypothèse avec tous les $|V|$ tokens possibles, calculer les $k \times |V|$ nouveaux scores
- Conserver les k meilleures hypothèses étendues
- Terminer quand toutes les hypothèses ont atteint <EOS> ou la longueur maximale

Pour éviter de favoriser artificiellement les séquences courtes (dont la log-probabilité cumulée est mécaniquement plus élevée), la pénalisation de longueur introduite par Google NMT est recommandée :

$$\text{score normalisé} = (\sum_t \log P(y_t | \dots)) / T^\alpha \text{ avec } \alpha \in [0.6, 0.8]$$

Stratégie	Complexité	Qualité (BLEU)	Vitesse
Glouton (k=1)	$O(T \times V)$	Sous-optimal	Très rapide
Beam Search k=3	$O(T \times 3 \times V)$	Bon compromis	3× plus lent
Beam Search k=5	$O(T \times 5 \times V)$	Meilleure qualité	5× plus lent
Recherche exhaustive	$O(V ^T)$	Optimal	Totalement impraticable

Tableau 30 : Comparaison des stratégies de décodage (Glouton vs Beam Search)

En pratique, $k = 3$ à 5 offre le meilleur compromis qualité/vitesse pour la traduction neuronale. Au-delà de $k = 10$, les gains de qualité deviennent marginaux.

3.12. Évaluation — Perplexité et Score BLEU

3.12.1. Deux Types de Métriques d'Évaluation

L'évaluation des modèles séquentiels s'appuie sur deux classes de métriques complémentaires :

- Métriques intrinsèques (indépendantes de la tâche) : la perplexité mesure la qualité du modèle de langage en tant que tel, sans référence à une tâche applicative. Elle peut être calculée sans annotations humaines supplémentaires.
- Métriques extrinsèques (spécifiques à la tâche) : le score BLEU mesure la qualité de la traduction par comparaison avec des références humaines. Elle requiert des traductions de référence annotées par des experts.

3.12.2. Score BLEU — Bilingual Evaluation Understudy

Le BLEU (Papineni et al., ACL 2002) est la métrique standard de la traduction automatique. Il mesure la précision des n -grammes entre l'hypothèse produite par le modèle et la ou les références humaines, avec une pénalité pour les traductions trop courtes (brevity penalty) :

$$\text{BLEU} = \text{BP} \times \exp(\sum_n w_n \log p_n) \text{ avec } w_n = 1/4 \text{ (BLEU-4)}$$

$$\text{BP} = 1 \text{ si } c > r, \exp(1 - r/c) \text{ si } c \leq r \text{ (pénalité de brèveté)}$$

où p_n est la précision clipée des n -grammes (qui empêche la triche par répétition du même n -gramme), c est la longueur de l'hypothèse et r est la longueur de référence. Le BLEU-4 (n allant de 1 à 4) est la variante la plus utilisée.

3.12.3. Limitations et Métriques Modernes

Le BLEU présente plusieurs limitations importantes qu'il faut connaître : il ne capture pas la sémantique (voiture et automobile sont considérés comme entièrement différents), il ne mesure pas la fluidité grammaticale directement, et il suppose une correspondance lexicale exacte avec la référence. Les métriques

modernes comme BERTScore et COMET répondent à ces limitations en utilisant des embeddings contextuels pour comparer l'hypothèse et la référence dans un espace sémantique continu, montrant une bien meilleure corrélation avec les jugements humains.

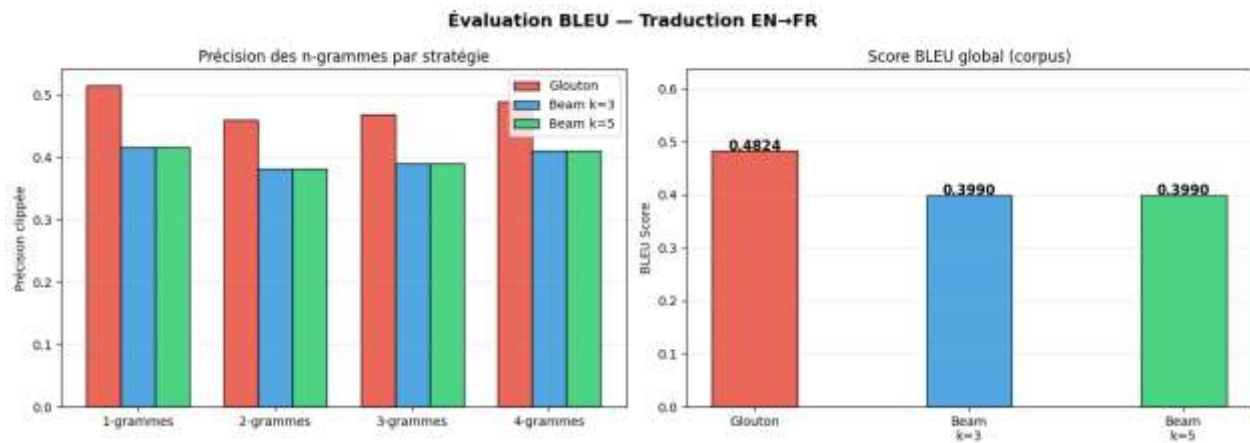


Figure 20 : Précision des n-grammes et score BLEU par stratégie de décodage

Cette figure compare la précision des n-grammes (1 à 4) et le score BLEU global pour le décodage glouton et le Beam Search à $k=3$ et $k=5$, confirmant que l'élargissement du faisceau améliore progressivement la qualité de traduction mesurée, au prix d'un coût computationnel accru.

3.13. Comparaison Globale des Architectures

Modèle	Atouts principaux	Limitations
RNN Vanilla	Léger, conceptuellement simple, base pédagogique	Gradient évanescent, mauvaise mémoire longue terme
LSTM	Excellent contrôle mémoire, robuste sur longues séquences	4× plus de paramètres, entraînement plus lent
GRU	Bon compromis précision/vitesse, 25% moins de params	Légèrement moins expressif sur très longues dépendances
RNN Profond ($L>1$)	Représentations hiérarchiques, abstraction croissante	Coût computationnel accru, entraînement plus délicat
RNN Bidirectionnel	Utilise passé ET futur, excellent pour l'encodage	Inadapté au décodage causal et à la génération
Seq2Seq	Modélise proprement la traduction $P(y x)$	Bottleneck du vecteur de contexte fixe, sans attention
Beam Search ($k>1$)	Meilleur BLEU que le décodage glouton	$k\times$ plus lent, sensible au réglage de α et k

Tableau 31 : Comparaison globale des architectures (RNN, LSTM, GRU, Seq2Seq...)

Ce tableau synthétise les compromis fondamentaux entre les différentes architectures. Le choix d'un modèle dépend de la tâche (génération vs encodage), des contraintes computationnelles (paramètres, vitesse d'inférence), et de la longueur typique des séquences à traiter.

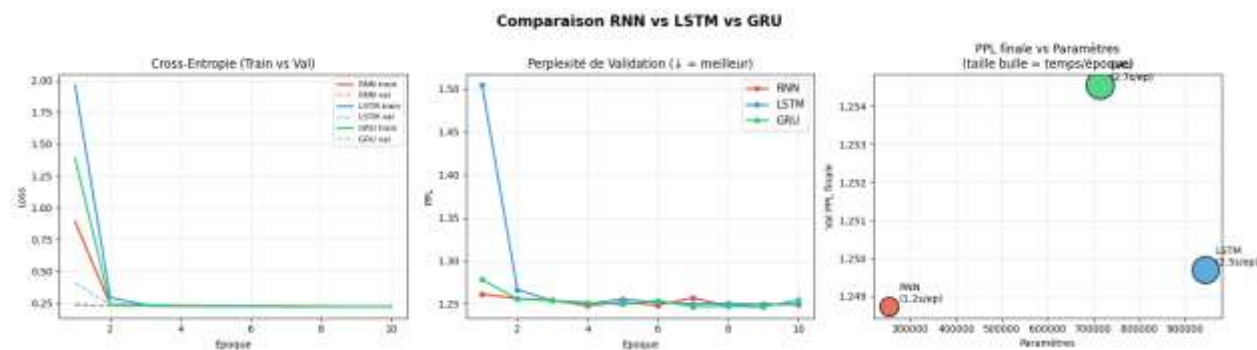


Figure 21 : Comparaison complète RNN vs LSTM vs GRU

Cette planche à trois graphiques synthétise l'entraînement comparatif des trois architectures : les courbes de perte confirment une convergence plus stable pour LSTM et GRU, la perplexité de validation montre leur supériorité sur le RNN vanilla, et le graphique paramètres/PPL/temps illustre le compromis entre coût computationnel et performance.

3.14. Synthèse Transversale — Du RNN au Seq2Seq

14.1 Du RNN Vanilla au LSTM/GRU — Résoudre le Problème de Mémoire

La règle de chaîne $P(w) = \prod_t P(w_t | w_{<t})$ définit la tâche de modélisation de séquence : prédire le prochain token conditionnellement à tout le contexte passé. Le RNN implémente cette récurrence via $h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$. En théorie, h_t encode l'intégralité de l'historique $w_1, \dots, w_t$. En pratique, le gradient évanescence l'en empêche : les informations distantes de plus de quelques pas de temps sont effectivement oubliées.

Le LSTM répond à cette limitation via son état de cellule c_t et sa mise à jour additive : $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. Cette opération additive crée une autoroute de gradient : le gradient $\partial c_t / \partial c_k$ n'implique plus un produit de matrices mais un produit de valeurs de porte d'oubli, potentiellement proches de 1. Les expériences empiriques confirment systématiquement la supériorité du LSTM et du GRU sur le RNN vanilla pour les séquences longues (Hochreiter & Schmidhuber, 1997 ; Cho et al., 2014).

3.14.1. Du Modèle de Langage au Seq2Seq — Généraliser à Deux Séquences

La traduction automatique exige $P(y | x)$ — une distribution conditionnelle sur deux espaces différents avec des longueurs asymétriques. L'architecture encodeur-décodeur découple proprement les deux phases : compréhension de la source ($x \rightarrow c$) et génération de la cible ($c, y_{<t} \rightarrow y_t$). Le Teacher Forcing accélère l'entraînement en guidant le décodeur avec les vrais tokens gold, mais crée un écart train/inférence (exposition

bias) qui peut dégrader la qualité à l'inférence. Le curriculum progressif (TF : 1.0 $\rightarrow$ 0.2) est la solution empiriquement robuste.

3.14.2. Qualité du Décodage — Beam Search vs Glouton

Le Beam Search améliore systématiquement le score BLEU par rapport au décodage glouton sur des séquences de longueur modérée. Sur un petit corpus comme Tatoeba, le bottleneck principal est la qualité du modèle encodeur-décodeur lui-même, pas la stratégie de décodage. $k = 3$ offre généralement le meilleur compromis qualité/coût computationnel.

3.14.3. Limites et Perspectives — Vers les Transformers

Malgré leurs apports considérables, les architectures récurrentes présentent deux limitations structurelles fondamentales qui ont motivé le développement des Transformers (Vaswani et al., 2017) :

- Bottleneck du vecteur de contexte : compresser une séquence source entière en un unique vecteur fixe $c \in \mathbb{R}^H$ est destructif pour les longues phrases. Le mécanisme d'attention de Bahdanau (2015) résout ce problème en permettant au décodeur d'accéder sélectivement à tous les états cachés de l'encodeur.
- Séquentialité fondamentale : le RNN, le LSTM et le GRU traitent les tokens un par un, de façon strictement séquentielle. Il est impossible de paralléliser sur la dimension temporelle, ce qui limite fortement l'efficacité sur GPU. Le Transformer (Vaswani et al., 2017) supprime complètement la récurrence au profit du mécanisme d'auto-attention (self-attention), permettant un traitement entièrement parallèle.
- Exposure bias : le curriculum learning, le Scheduled Sampling et le Minimum Risk Training sont des solutions partielles, mais aucune n'élimine complètement le problème.

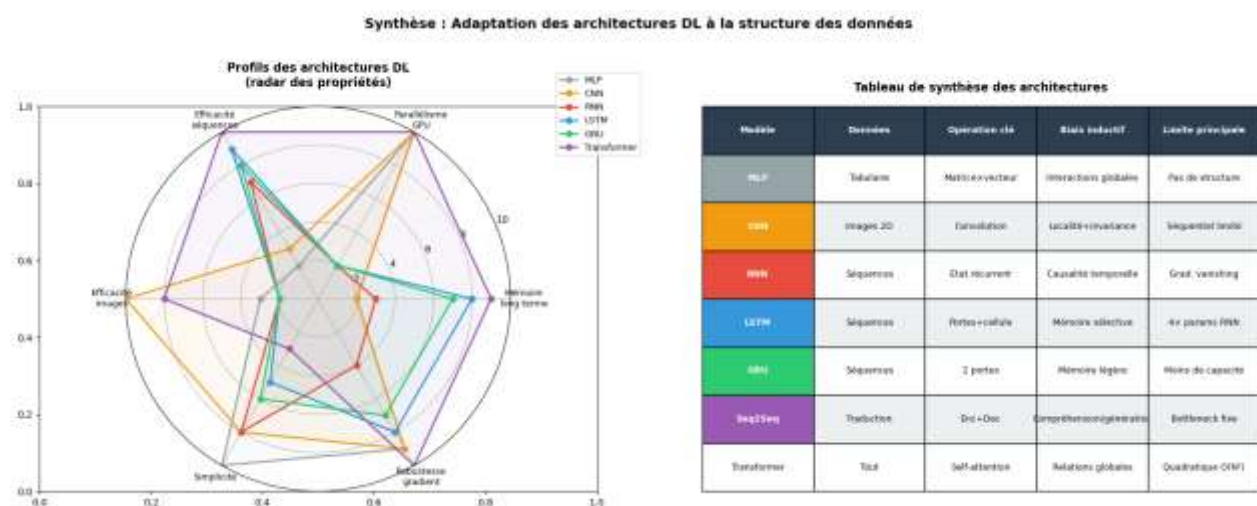


Figure 22 : Synthèse finale : profils radar et tableau comparatif des architectures

Cette double visualisation résume l'ensemble du chapitre : le diagramme radar positionne chaque architecture selon six critères clés (mémoire long terme, parallélisme, robustesse au gradient...), tandis que le tableau récapitulatif associe à chaque modèle sa structure de données cible, son opération clé, son biais

inductif et sa limite principale, offrant une vue d'ensemble de la progression MLP → CNN → RNN → Seq2Seq → Transformer.

3.15. Conclusion

Les RNN, LSTM et GRU ont permis d'améliorer le traitement des données séquentielles, tandis que les modèles Seq2Seq ont ouvert la voie à des applications comme la traduction automatique. Malgré leurs limites sur les longues séquences, ces architectures restent essentielles pour comprendre les fondements du Deep Learning appliqué au traitement du langage.

Conclusion Générale

Le Deep Learning occupe aujourd'hui une place centrale dans le développement des systèmes d'intelligence artificielle modernes. La diversité des problématiques rencontrées dans des domaines tels que la classification, la vision par ordinateur ou le traitement automatique du langage a conduit à l'émergence d'architectures spécialisées, chacune répondant à des besoins spécifiques. Dans ce contexte, ce projet avait pour objectif d'étudier, d'implémenter et de comparer trois grandes familles de réseaux de neurones : les perceptrons multicouches (MLP), les réseaux de neurones convolutionnels (CNN) et les réseaux de neurones récurrents (RNN).

La première partie a permis d'aborder les fondements du Deep Learning à travers les MLP appliqués à des données tabulaires. Cette étude a mis en évidence les mécanismes essentiels de l'apprentissage supervisé, notamment la propagation avant, la rétropropagation du gradient et l'optimisation des paramètres du modèle. Bien qu'efficaces sur des données vectorisées, les MLP présentent certaines limites lorsqu'ils sont confrontés à des données possédant une structure spatiale ou temporelle complexe.

La deuxième partie, consacrée aux CNN, a démontré l'intérêt des opérations de convolution et de pooling dans le traitement des images. Les expérimentations réalisées sur le jeu de données Fashion-MNIST ont montré que ces architectures sont capables d'extraire automatiquement des caractéristiques pertinentes, conduisant à des performances supérieures à celles des approches entièrement connectées dans les tâches de classification visuelle. L'analyse des cartes de caractéristiques et l'étude des hyperparamètres ont également permis de mieux comprendre le fonctionnement interne des modèles convolutionnels.

Enfin, la troisième partie a exploré les architectures dédiées aux données séquentielles. L'étude des RNN, des LSTM et des GRU a mis en évidence les défis liés à la modélisation des dépendances temporelles ainsi que les solutions proposées pour atténuer les problèmes de disparition ou d'explosion du gradient. La mise en œuvre d'un modèle Seq2Seq appliqué à une tâche de traduction automatique a illustré la capacité des réseaux récurrents à traiter des séquences de longueur variable et à générer des sorties cohérentes. Les métriques d'évaluation, telles que la perplexité et le score BLEU, ont permis d'apprécier objectivement la qualité des modèles développés.

Au-delà des résultats obtenus, ce projet a permis de renforcer des compétences essentielles en Deep Learning, notamment en conception de modèles, en utilisation de PyTorch et en analyse des performances. Les expérimentations réalisées ont montré que le choix d'une architecture dépend fortement de la nature des données et des objectifs visés. En perspective, ce travail pourrait être enrichi par l'étude d'architectures plus avancées, telles que les ResNet ou les Transformers, afin d'approfondir la compréhension des techniques modernes d'intelligence artificielle.

Références

- **Cybenko (1989)** : démonstration du théorème d'approximation universelle pour les réseaux à une couche cachée.
- **Hornik (1991)** : extension du théorème d'approximation universelle à des classes plus générales de réseaux de neurones.
- **LeCun et al. (1989)** : première démonstration empirique de l'efficacité des réseaux de neurones convolutionnels.
- **LeCun et al. (1998)** : proposition de l'architecture **LeNet-5** pour la reconnaissance automatique des chiffres manuscrits.
- **Srivastava et al. (2014)** : introduction de la technique de régularisation **Dropout**.
- **Glorot & Bengio (2010)** : développement de la méthode d'initialisation des poids **Xavier**.
- **Ioffe & Szegedy (2015)** : introduction de la **Batch Normalization** pour stabiliser et accélérer l'entraînement.
- **Kingma & Ba (2014/2015)** : proposition de l'optimiseur **Adam**, largement utilisé en Deep Learning.
- **Xiao et al. (2017)** : création du jeu de données **Fashion-MNIST** comme alternative à MNIST.
- **Hubel & Wiesel (1981)** : travaux sur les cellules simples du cortex visuel V1, ayant inspiré les CNN (Prix Nobel de physiologie ou médecine).
- **Hochreiter & Schmidhuber (1997)** : introduction des réseaux **LSTM (Long Short-Term Memory)**.
- **Cho et al. (2014)** : proposition des unités récurrentes **GRU (Gated Recurrent Unit)**.
- **Sutskever, Vinyals & Le (2014)** : développement de l'architecture **Seq2Seq (Sequence-to-Sequence)**.
- **Bahdanau et al. (2015)** : introduction du **mécanisme d'attention** dans les modèles Seq2Seq.
- **Vaswani et al. (2017)** : proposition des **Transformers**, fondés sur le mécanisme de self-attention.
- **Papineni et al. (2002)** : développement du score **BLEU** pour l'évaluation automatique des traductions.
- **Google Neural Machine Translation (GNMT)** : introduction de la **pénalisation de longueur** pour améliorer le Beam Search en traduction automatique.